

Chapter 5: Graph Visualization

Prof. Dr.-Ing. Thomas Schultz

URL: <http://cg.cs.uni-bonn.de/schultz/>

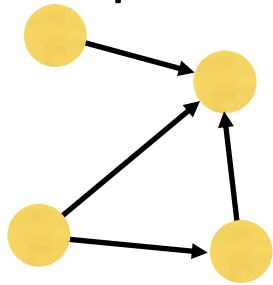
E-Mail: schultz@cs.uni-bonn.de

Office: Friedrich-Hirzebruch-Allee 6, Room 2.117

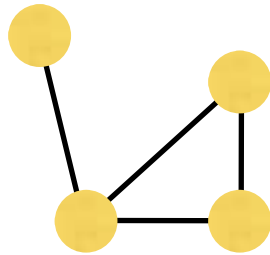
May 13, 2026

Reminder: Trees and Graphs

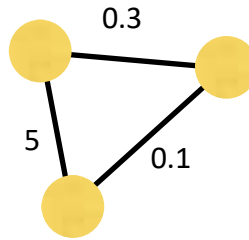
- **Graphs** $G = (V, E)$ model relations (edges E) between objects (nodes V)
- **Trees** are connected acyclic graphs
 - often represent a hierarchical structure



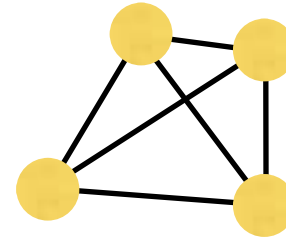
A directed graph



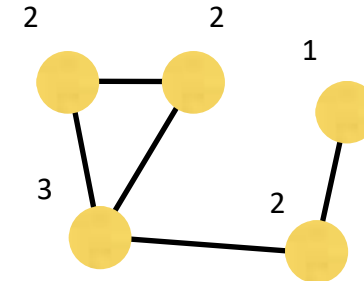
An undirected graph



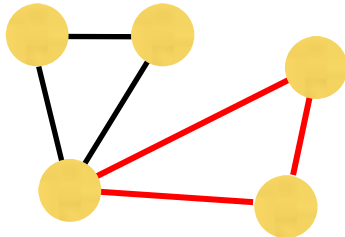
Weighted



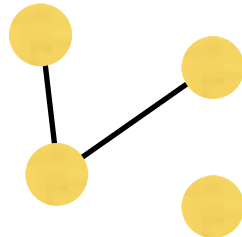
Complete



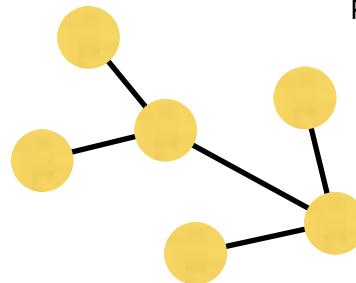
Node degrees



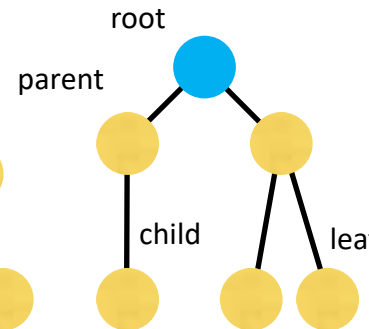
A cycle



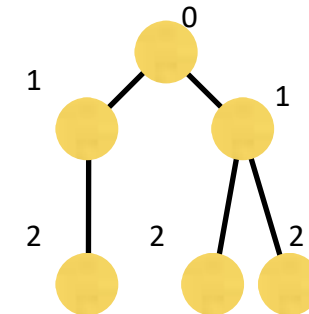
An acyclic graph



A connected acyclic graph,
a.k.a. a **tree**



A rooted tree
or hierarchy

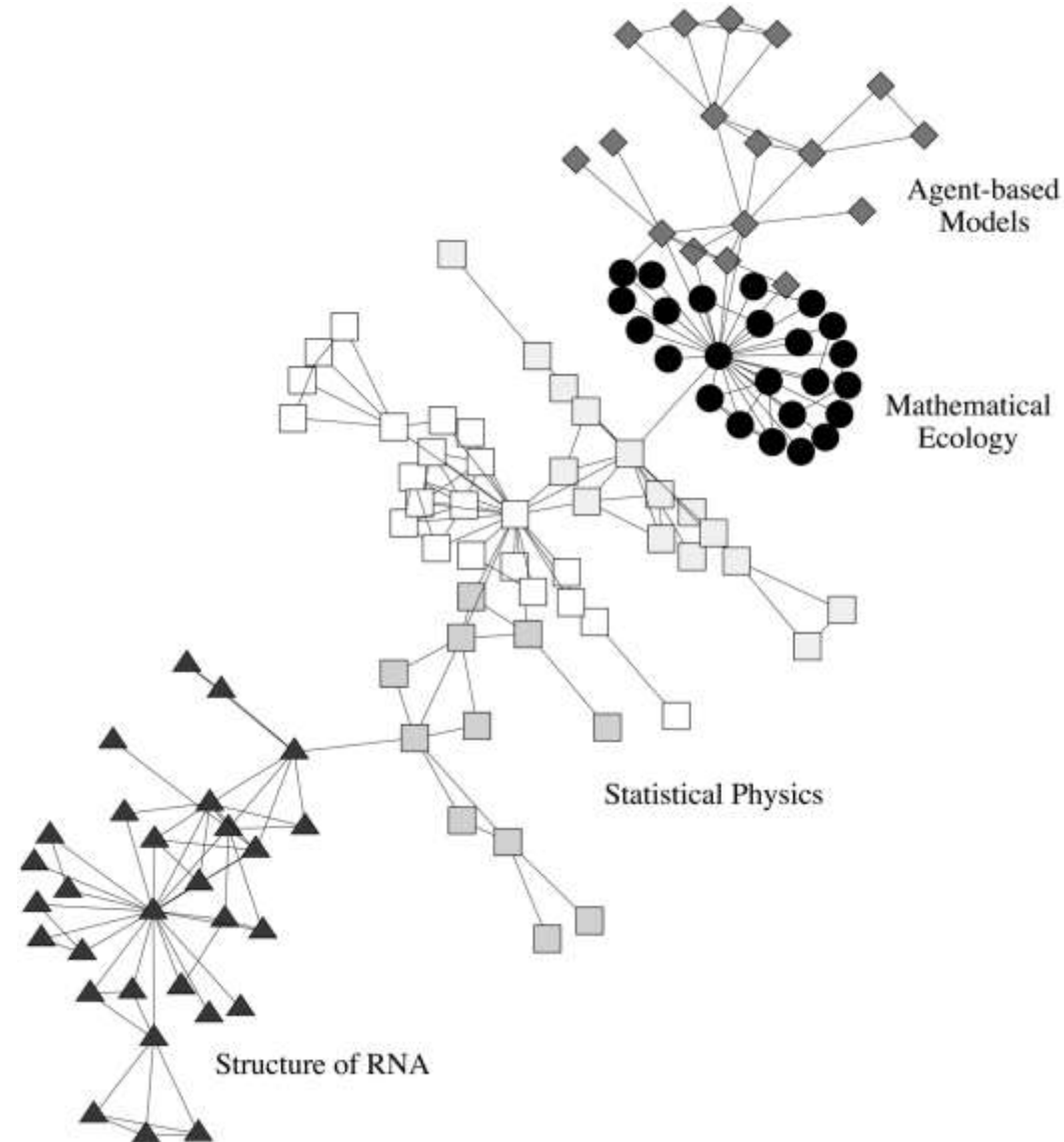


Node depths

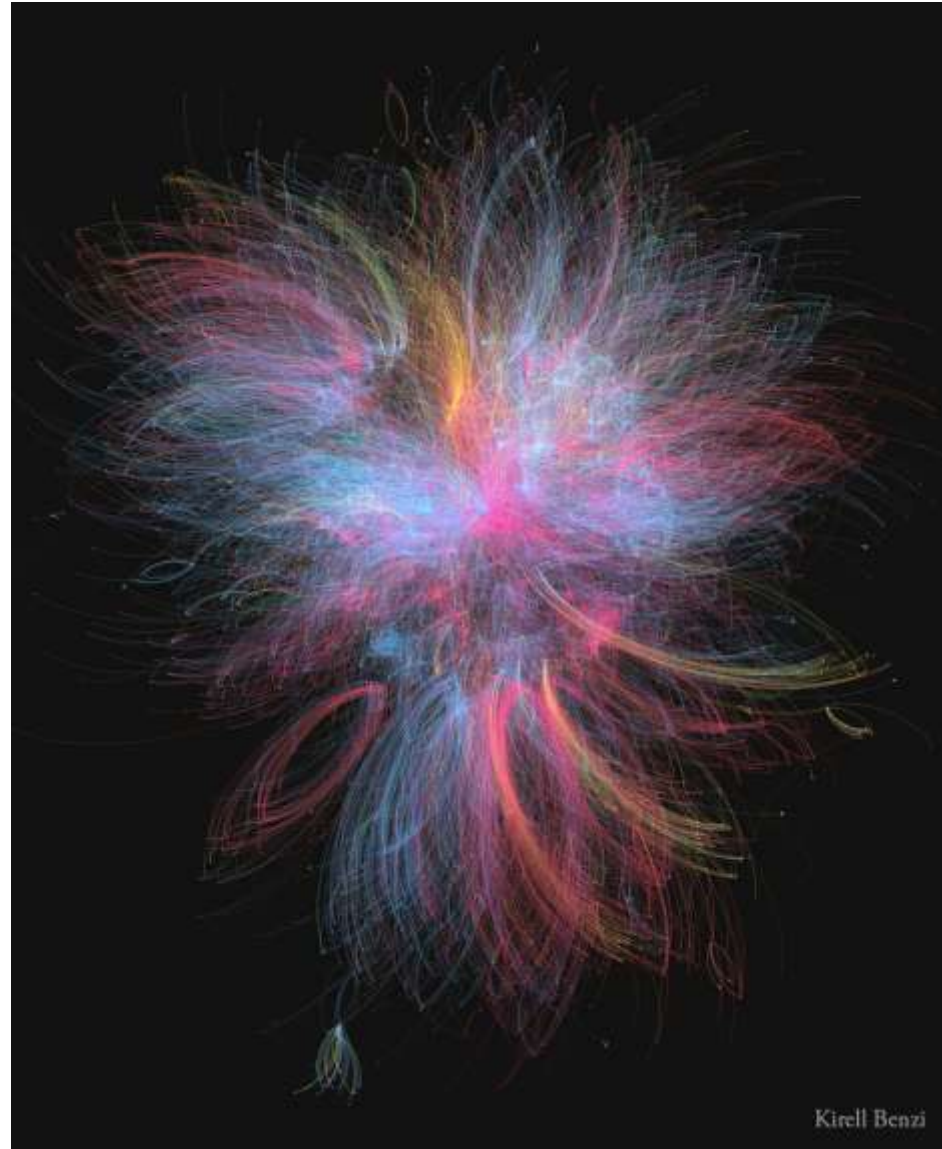
Goal of This Chapter

- Primary concern of **graph visualization** is the spatial layout of nodes and edges
- This does not necessarily imply node-link diagrams! Different representations support specific tasks:
 - connectivity, path-following
 - network distance
 - clustering
 - hierarchy
 - additional attributes

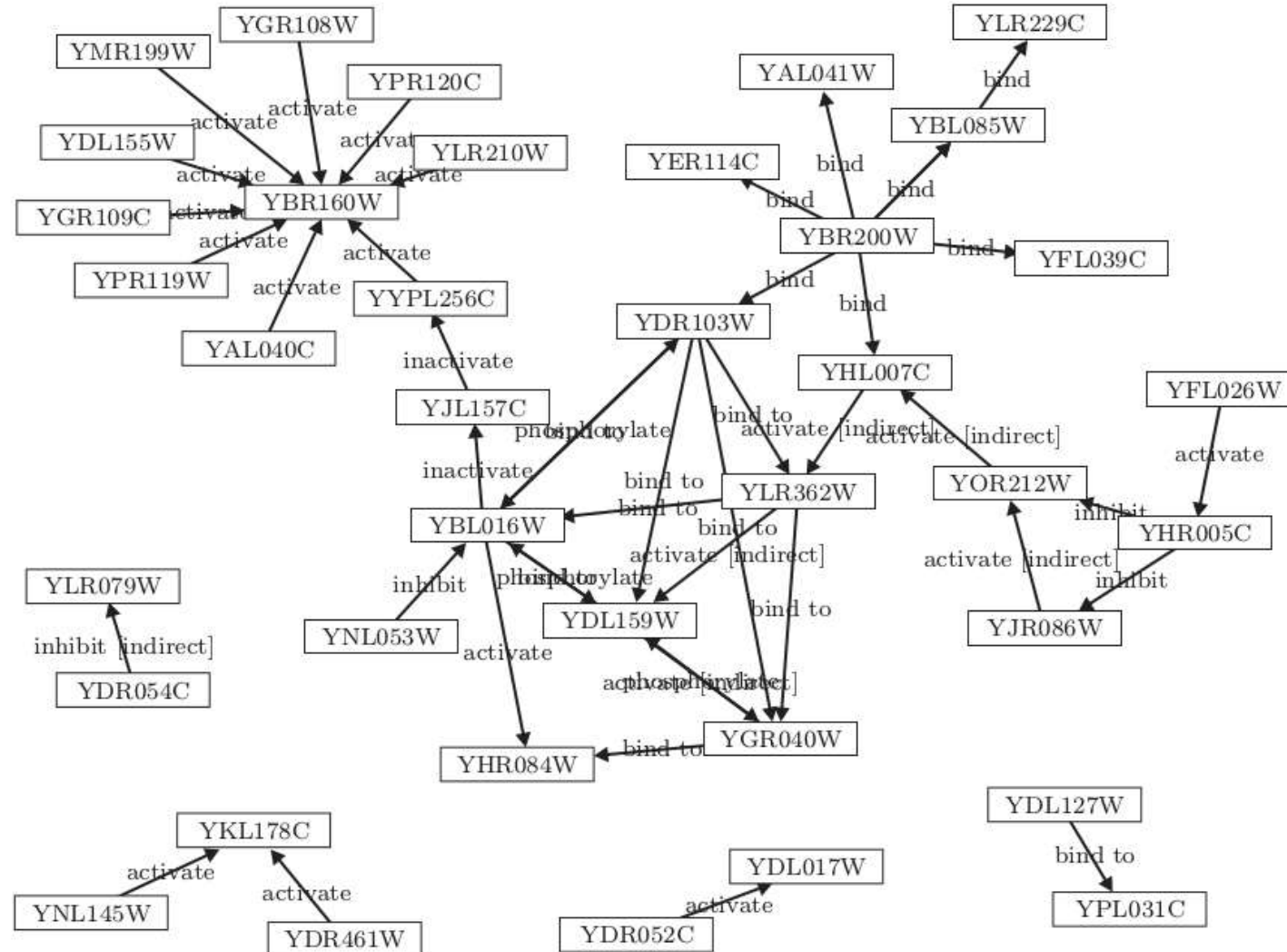
Example: Collaboration Network



Example: The Dark Side and the Light



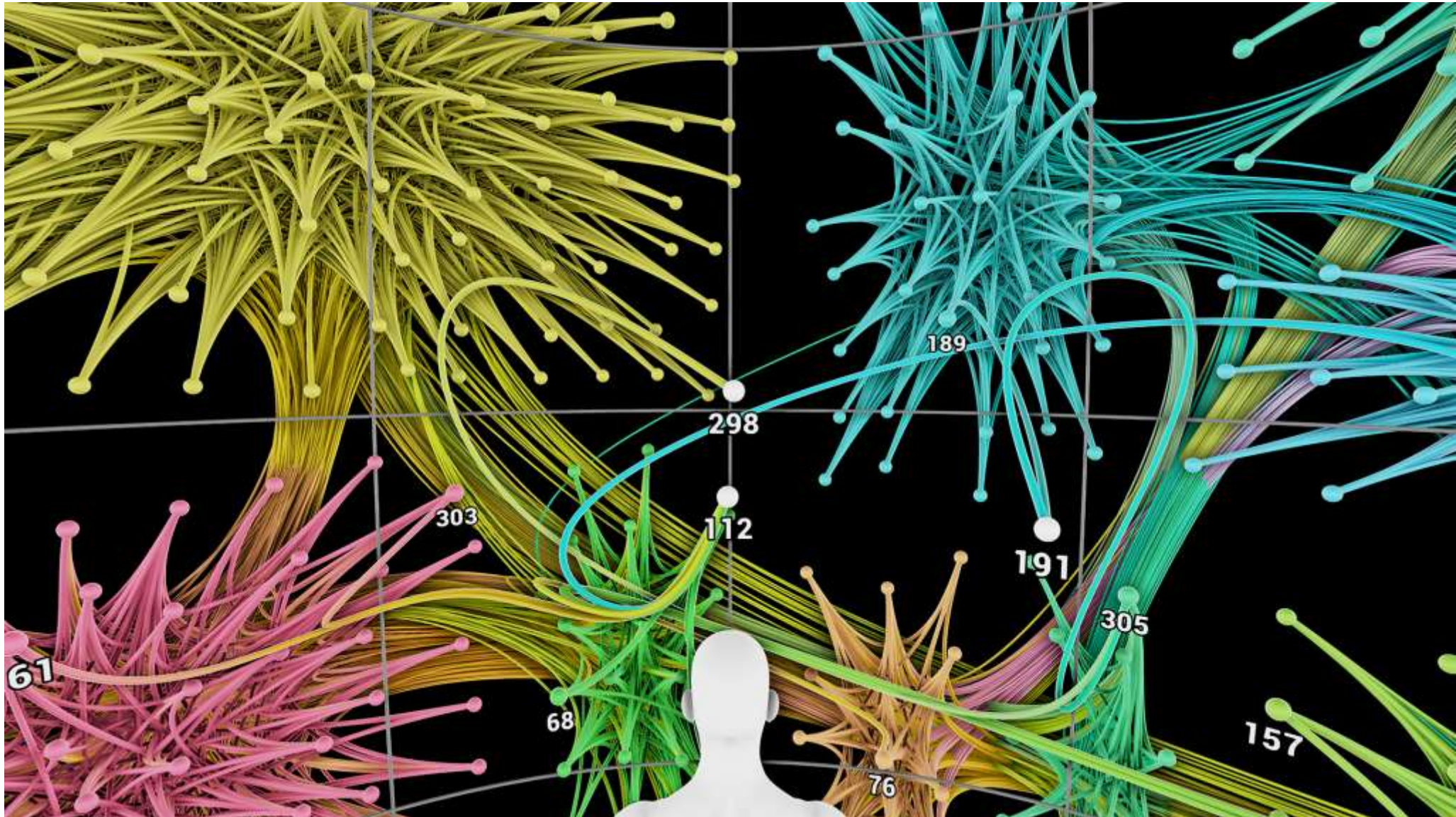
Example: Protein-Protein Interactions in Yeast





facebook

Example: Immersive Graph Visualization



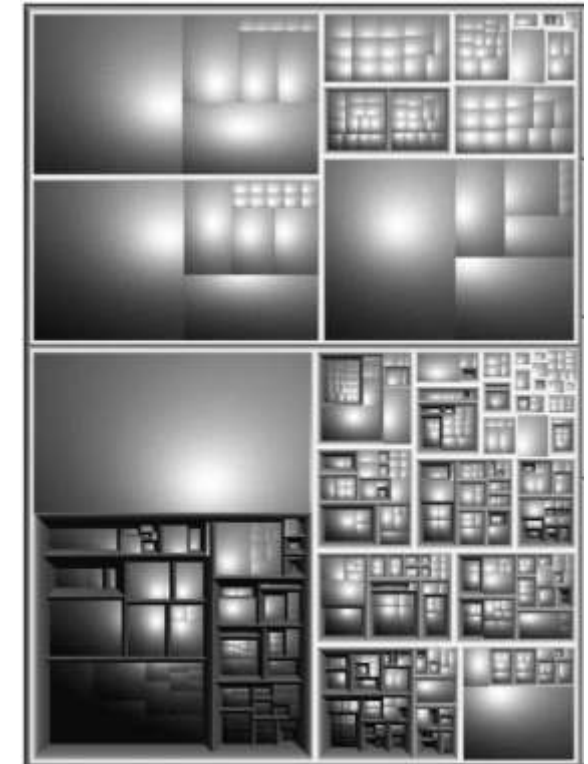
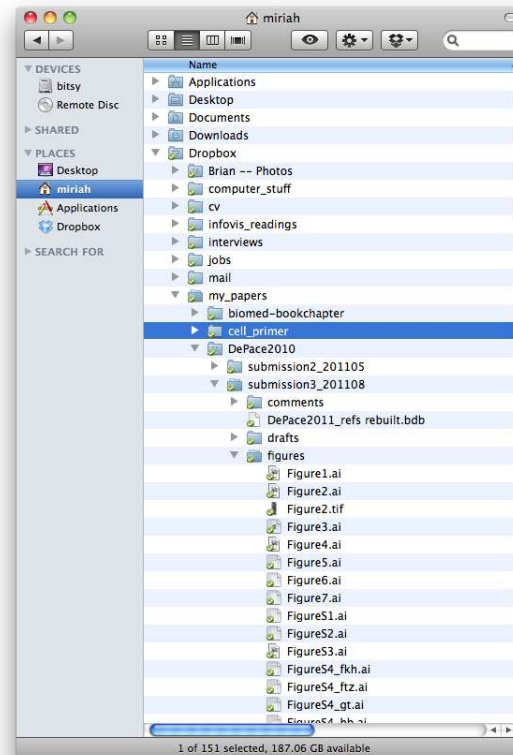
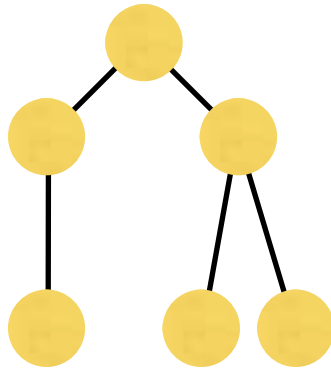
Section 5.1:

Visualizing Trees / Hierarchies

Visualizing Rooted Trees

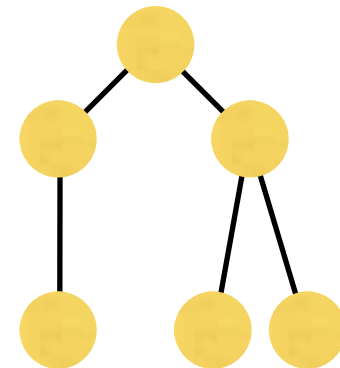
We will consider three approaches:

- node-link diagrams
- indentation
- enclosure (treemaps)



Visualizing Trees: Node-Link Diagrams

- In a **node-link diagram**, nodes are distributed in space, and are connected by straight lines or curves
 - In trees, 2D space is often used to break apart breadth and depth
- Frequent **design goals**:
 - Nodes at the same depth share the same vertical position
 - Horizontal whitespace reinforces hierarchy
 - Minimize required area
 - Minimize total length of edges
 - Achieve good aspect ratio

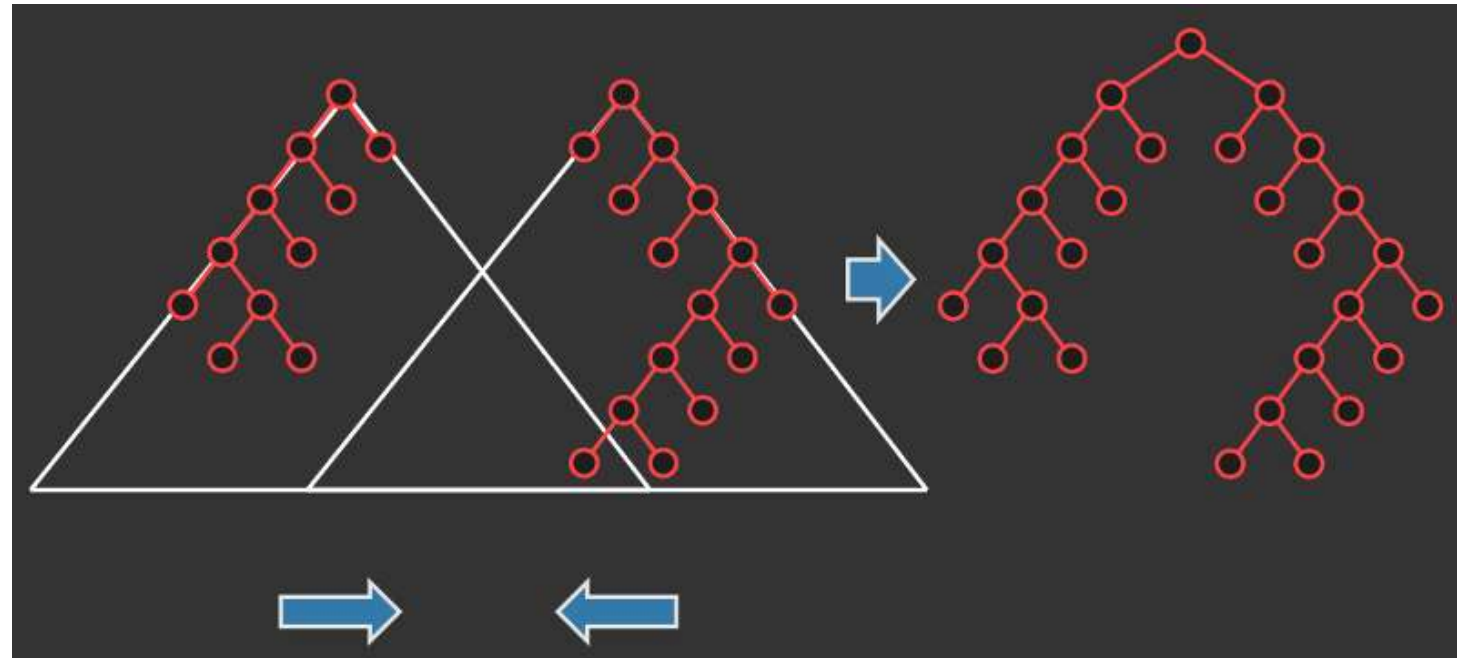


Aesthetics of Reingold-Tilford

- *Tidier Drawings of Trees* [Reingold/Tilford 1981]
 - Formulation for binary trees, can be generalized
- **Aesthetic Goals:**
 - Nodes at the same depth should be aligned
 - Maintain the relative ordering of left and right subtrees
 - Parent should be centered over the children
 - A tree and its mirror image should be drawn as reflections of each other
 - A subtree should be drawn the same way regardless of where it occurs in the tree
- **Quiz:** Which algorithmic approach could we use?

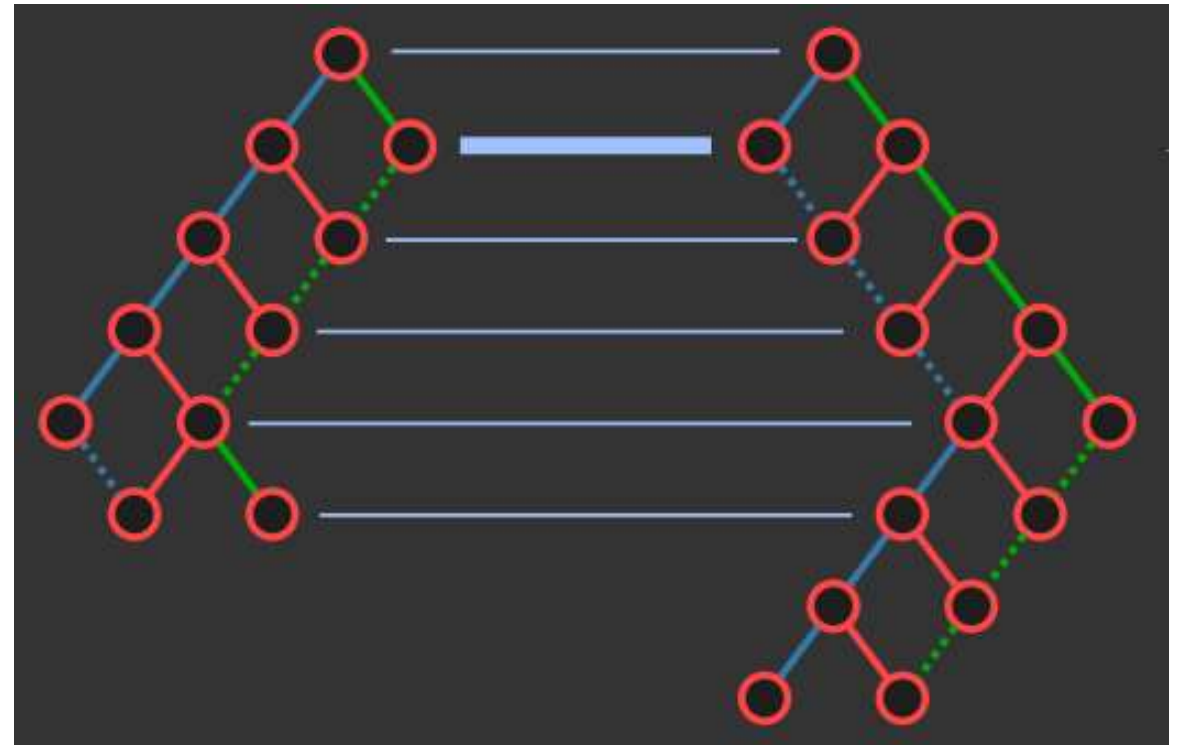
Reingold-Tilford: Recursive Construction

- Assume left and right subtrees have already been drawn
- Shift them to a fixed horizontal distance
- Center parent between them



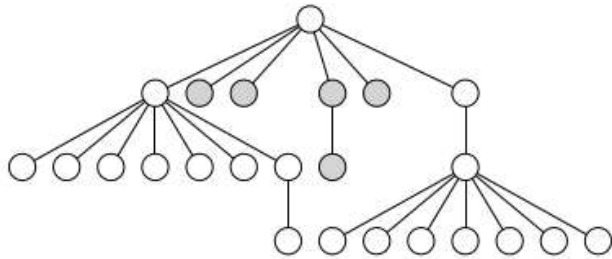
Reingold-Tilford: Threading

- Finding the correct distance requires traversal of the contours of each subtree
- If a contour node at depth k is a leaf, store a pointer, referred to as a *thread*, to the corresponding contour node at depth $k+1$
- Add a thread whenever two subtrees of different height are combined

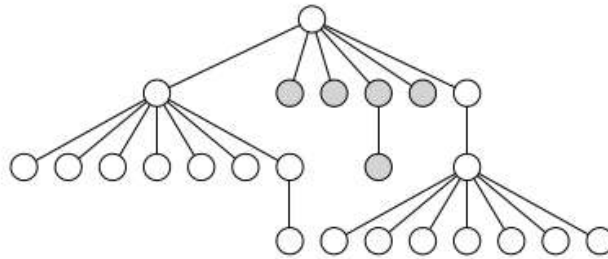


Reingold-Tilford for Unbounded Trees

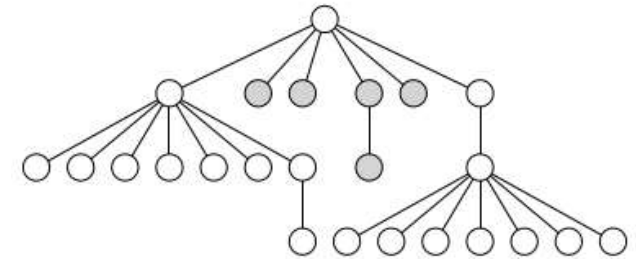
- Generalizing Reingold-Tilford to trees with **unbounded vertex degrees**:
 - Simply processing siblings left-to-right piles small sub-trees to the left (a)
 - Can be avoided by averaging positions with those from processing right-to-left (b), but small subtrees still cluster together



(a)



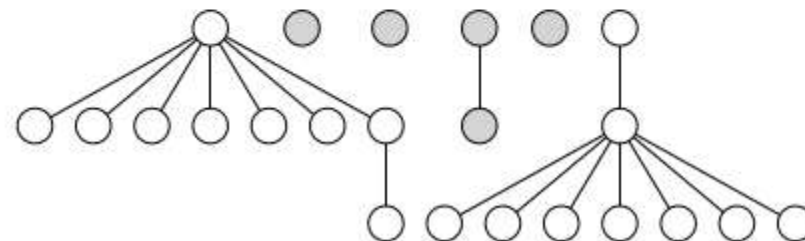
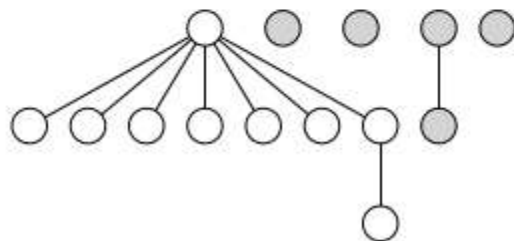
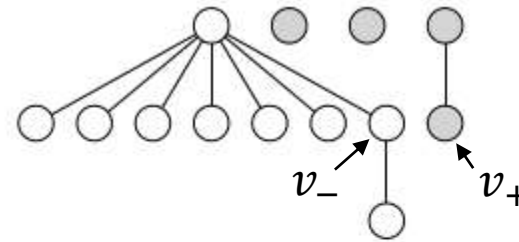
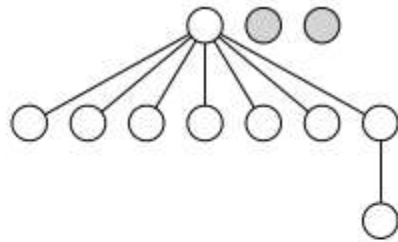
(b)



(c)

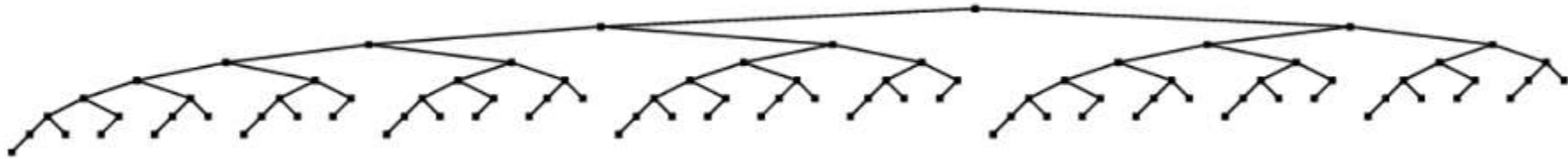
Walker's Algorithm

- **Walker's algorithm** for drawing trees with unbounded vertex degrees [Walker 1989]:
 - Start by placing siblings as far to the left as possible
 - Whenever v_- and v_+ along a contour require a shift, distribute it evenly to all subtrees in between

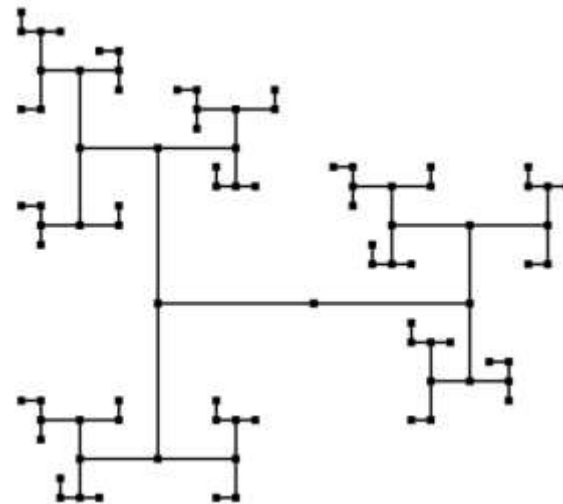


Reingold-Tilford: Pros and Cons

- The Reingold-Tilford algorithm is
 - easy to understand and implement, but
 - can lead to poor aspect ratios:

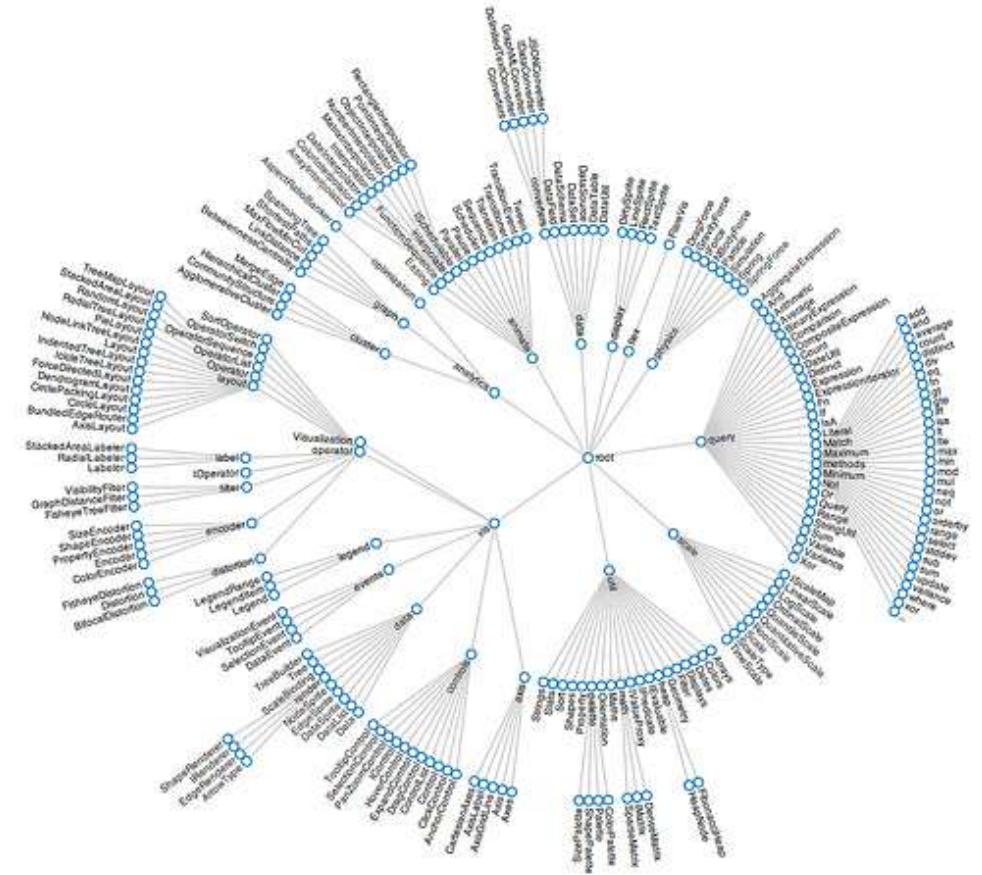


- alternative, non-level based layout of the same tree:



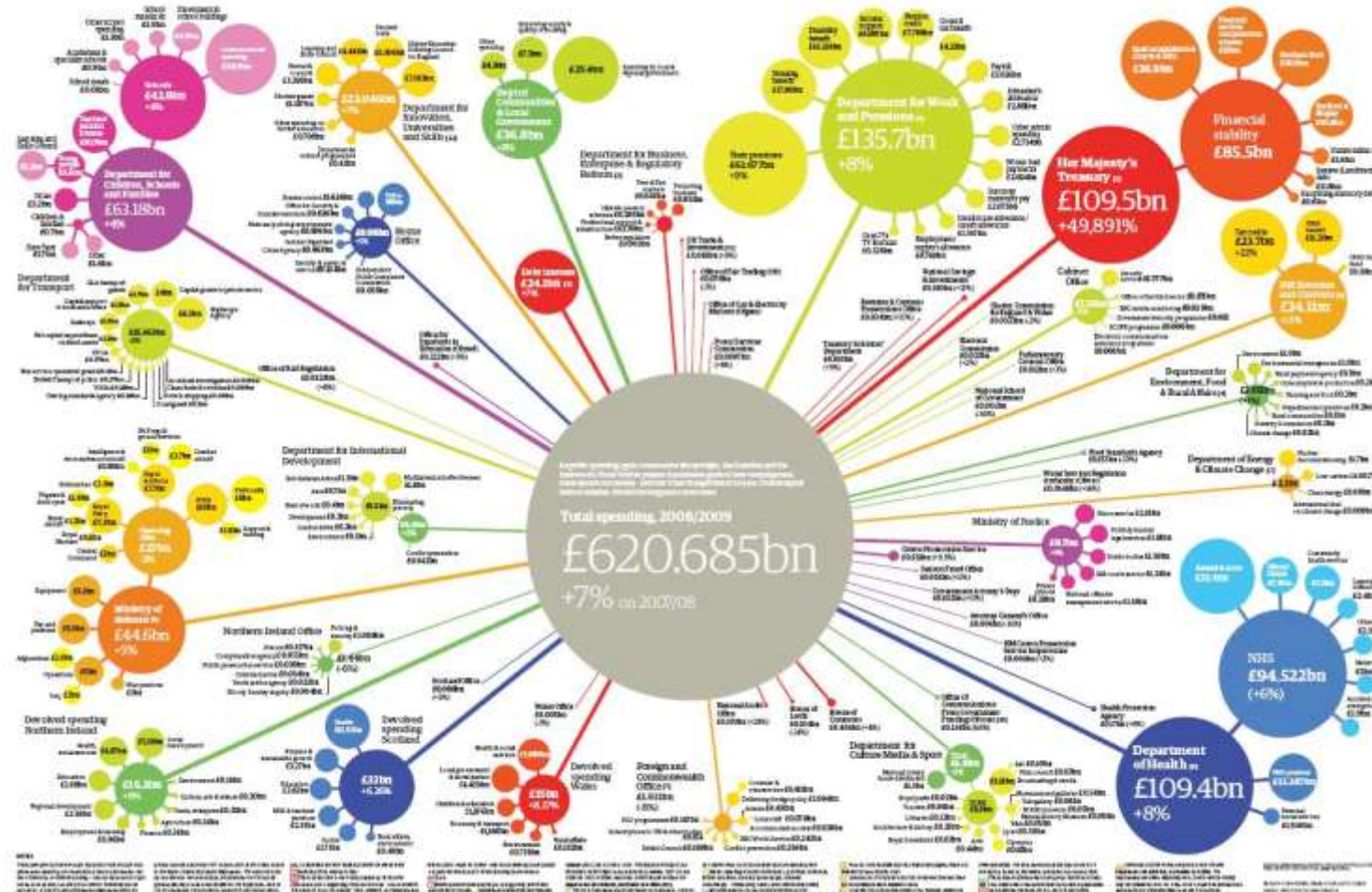
Node-Link Diagrams: Radial Layout

- node-link diagram in polar coordinates
- radius encodes depth with root in center
- angular sectors assigned to subtrees
- Reingold-Tilford can be applied



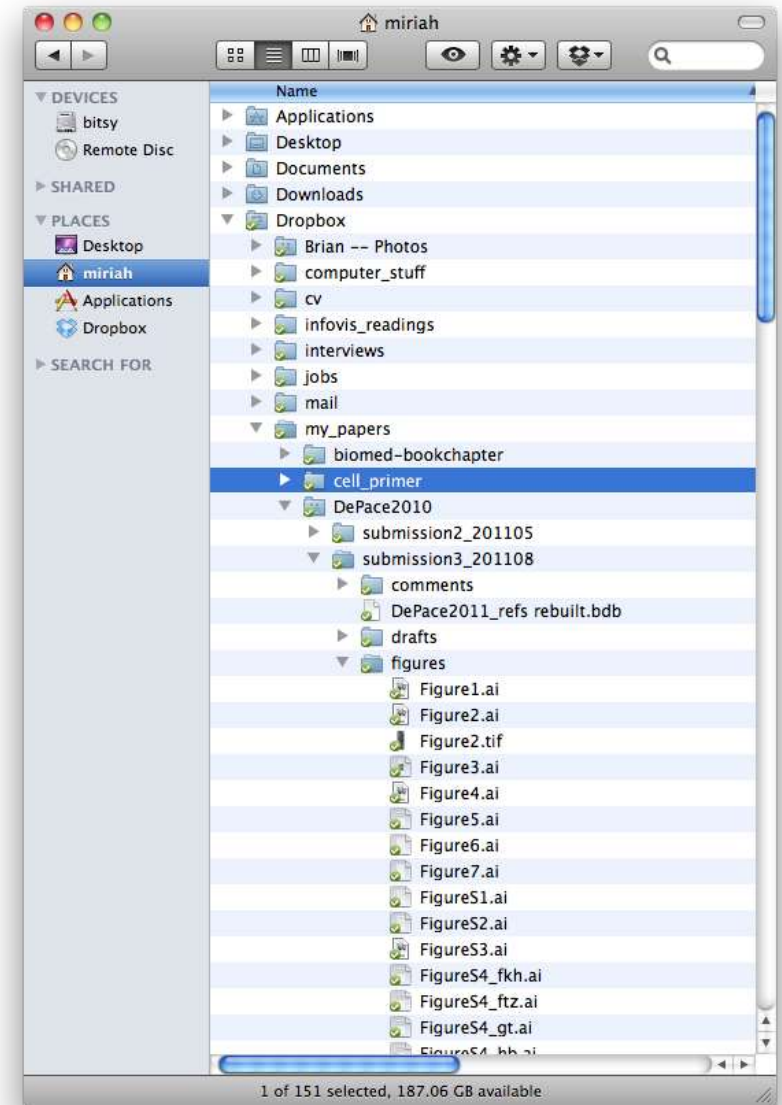
Node-Link Diagrams: Bubble Tree Layout

- *Variant:* Each inner node becomes the center of all its children, makes it easy to distinguish subtrees



Visualizing Trees: Indentation

- place all items along vertically spaced rows
- indentation used to show parent/child relationships
- commonly used as a component in user interfaces
- breadth and depth contend for space
- often requires a great deal of scrolling



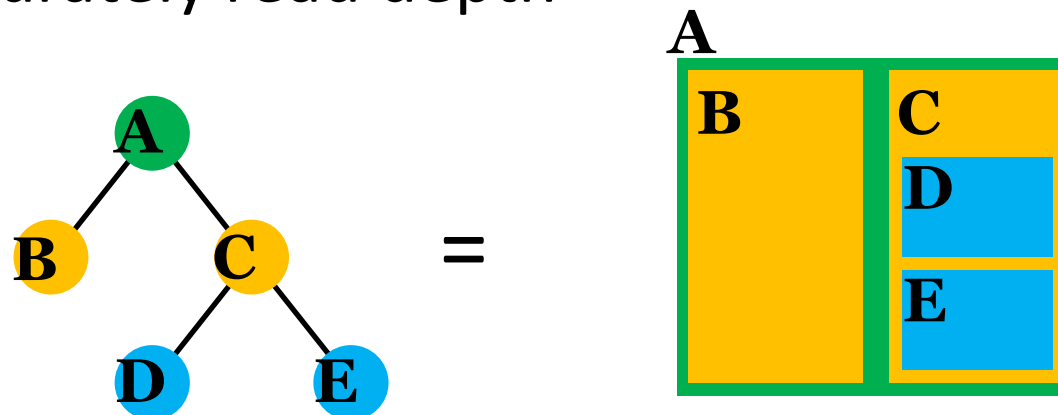
Visualizing Trees: Indentation

- Trees are usually large and unbalanced
- How to use the rectangular screen space optimally?



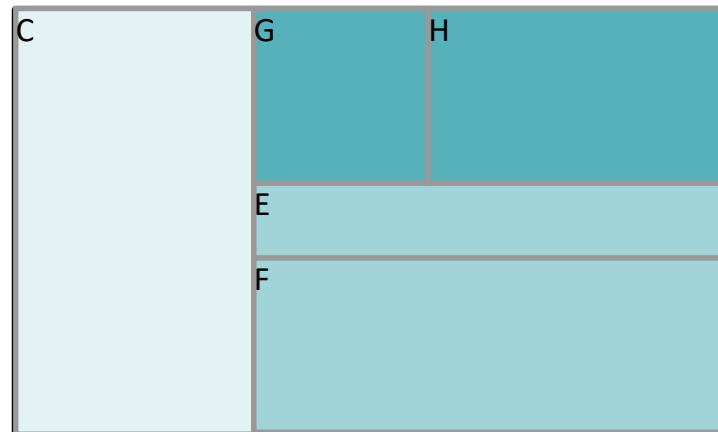
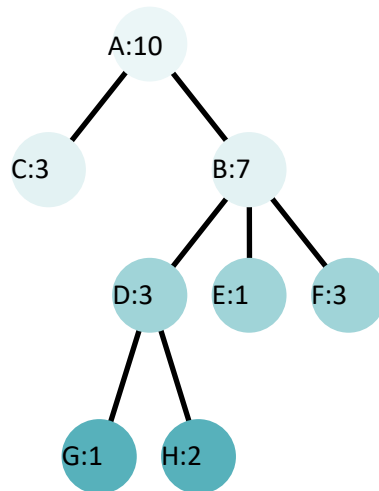
Visualizing Trees: Treemaps

- **Treemaps** encode structure using spatial enclosure
- **Benefits:**
 - provides single view of entire tree
 - easy to tell the “size” of node
 - easy to encode additional attributes (color)
- **Problems:**
 - difficult to accurately read depth



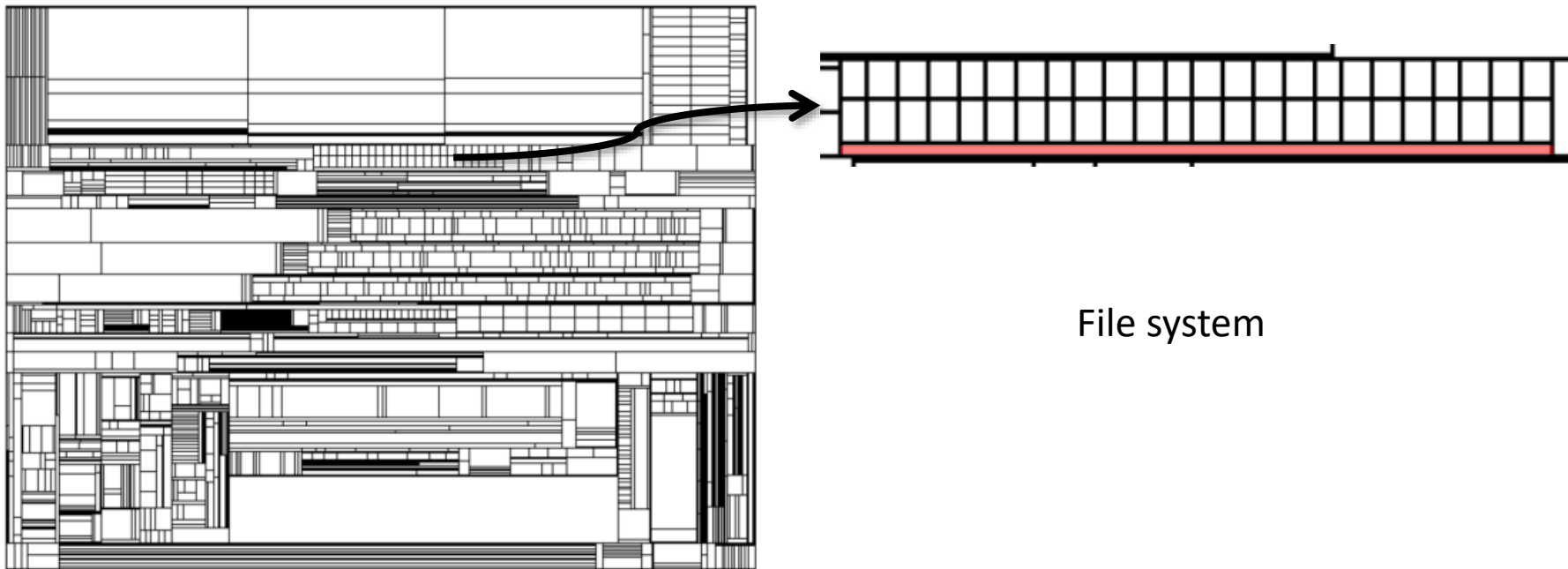
Treemaps: Basic Algorithm

- A basic **treemap algorithm** recursively fills available space based on a size metric for nodes
 - Simplest case: Alternate between horizontal and vertical splits



Treemaps: Problem of Aspect Ratios

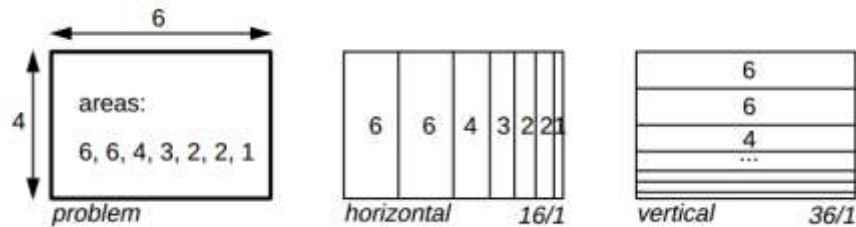
- **Problem:** Naïve splitting can lead to rectangles with poor aspect ratios
 - finding an optimal splitting is NP-hard



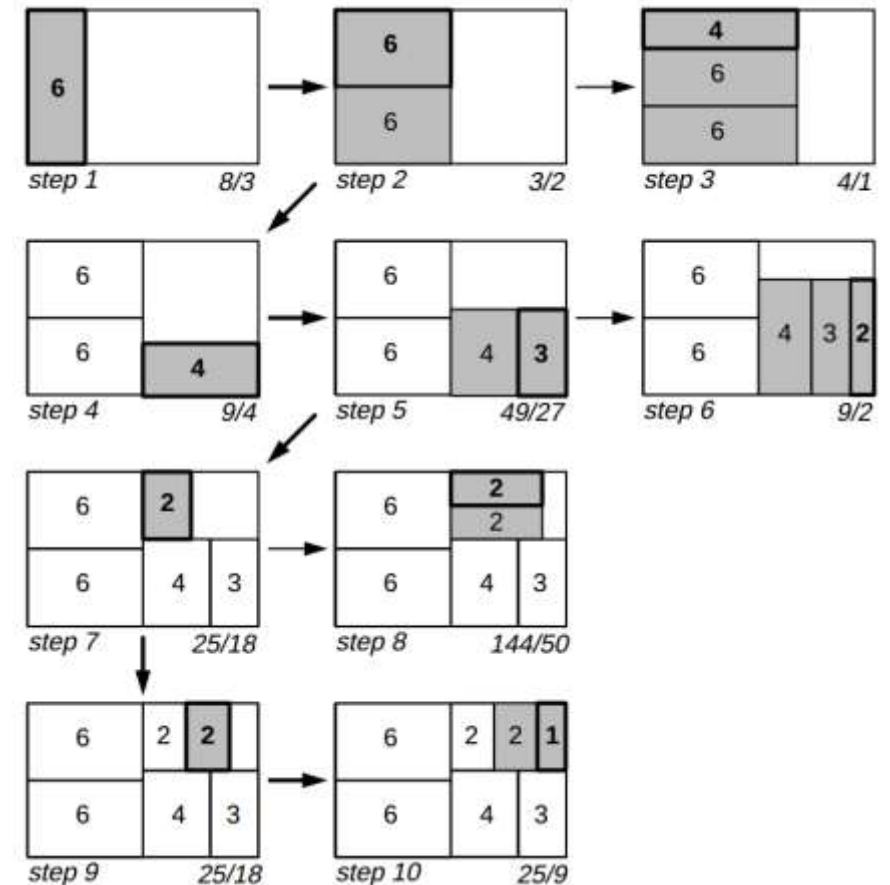
Bruls, Mark, Kees Huizing, and Jarke J. Van Wijk.
"Squarified treemaps." Data Visualization. 2000.

Visualizing Trees: Squarified Treemaps

- **Squarified treemaps:** greedy heuristic to favor squares over elongated rectangles

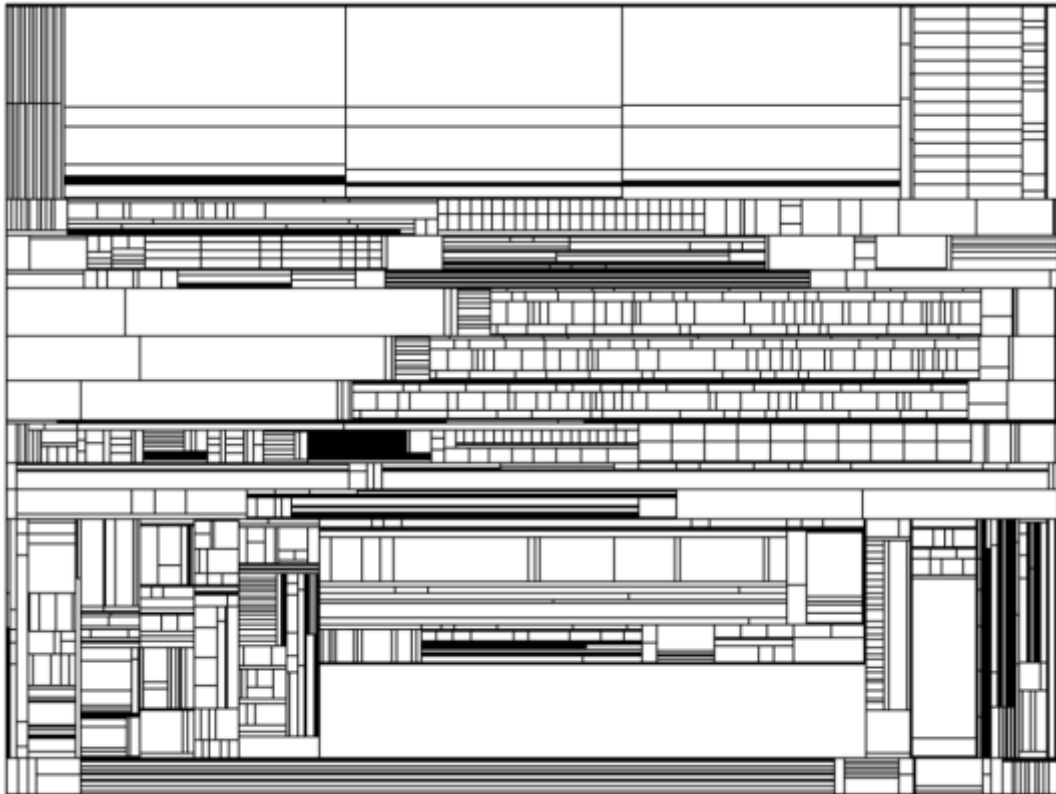


- Sort sibling nodes by size, insert them in descending order
- Split the wider dimension first
- Continue adding nodes while the aspect ratio improves

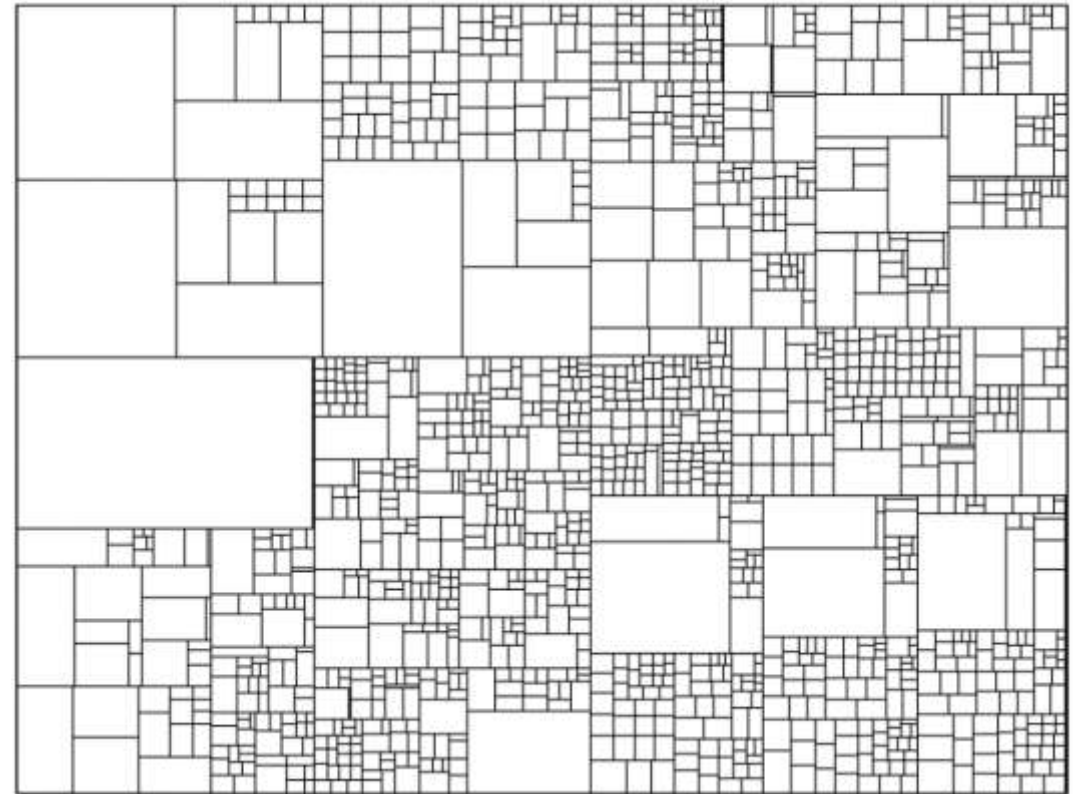


Example Result: Squarified Treemap

- File system example from [Bruls et al. 2000]
 - **Drawback:** Hierarchical structure even more difficult to perceive



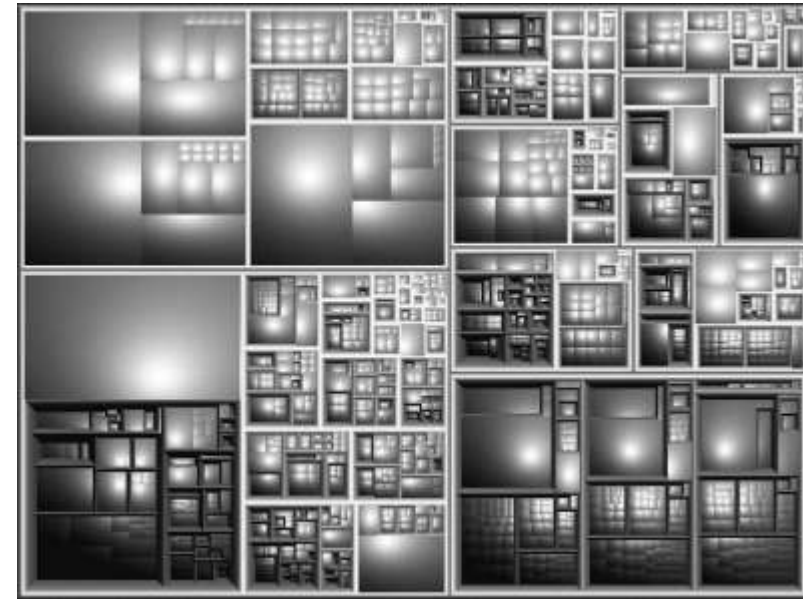
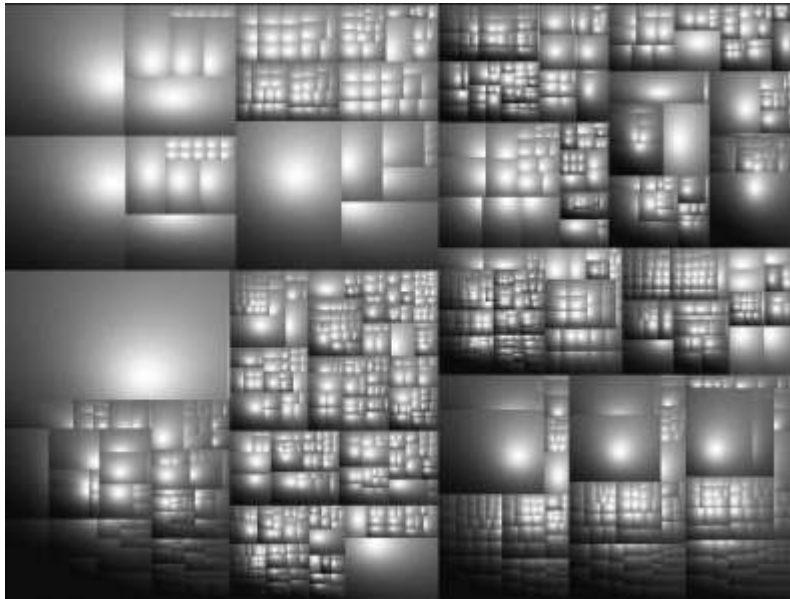
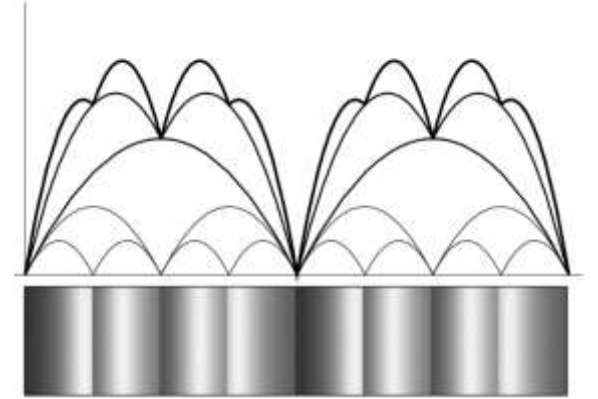
Traditional Treemap



Squarified Treemap

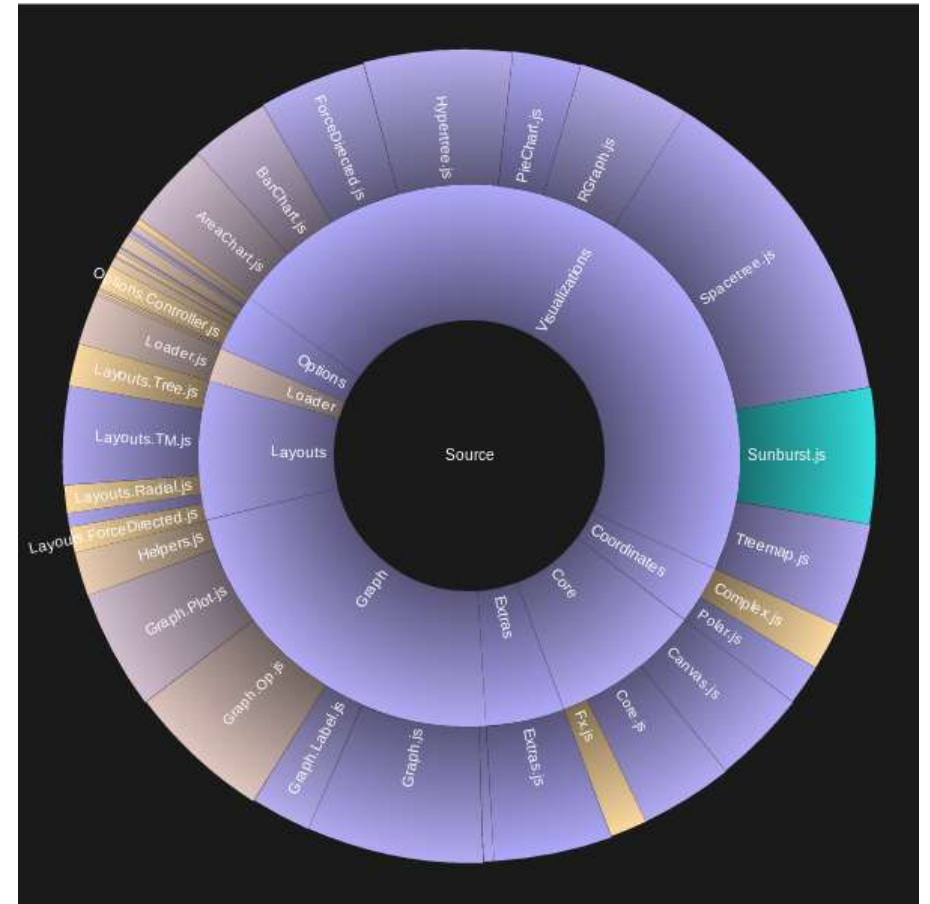
Visualizing Trees: Cushion Shading

- **Cushion shading** supports the perception of hierarchies
- $I = I_a + I_d \max(0, \mathbf{n} \cdot \mathbf{l})$
- Can add **frames and interaction**



Sunburst Displays

- **Sunburst displays**
 - Root at center, nested rings around it
 - Space-filling like treemaps
 - Layout similar to radial node-link diagrams



Summary: Visualizing Trees / Hierarchies

- **Main strategies for visualizing trees:**

- Node-link diagrams

- Reingold-Tilford
 - Walker's algorithm
 - Radial layouts

- Indentation

- Containment (treemaps)

- Squarification
 - Cushions and frames
 - Sunburst displays

Section 5.2: Force-directed Graph Layouts and Matrix Views

Idea: Force-Directed Layouts

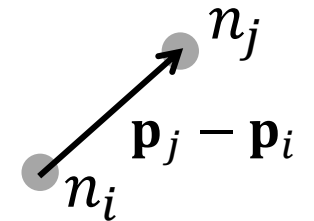
- **Goals:** To keep a node-link diagram readable, we should place groups of strongly connected nodes close to each other, while preserving a minimum distance between nodes
- **Idea:** Model graph as a spring system
 - Edges are modeled as springs
 - To avoid overlaps, nodes are modeled to electrically repel each other
 - Final positions are found by simulating the resulting forces

Definition of Specific Forces

- Different **force terms** have been explored
- Widely used example:
 - *Spring force* that node n_j exerts on n_i if an edge connects them

- node positions $\mathbf{p}_i$, natural spring length s_{ij} , tension k_{ij}

$$\mathbf{f}(\mathbf{p}_i, \mathbf{p}_j) = k_{ij} (\|\mathbf{p}_j - \mathbf{p}_i\| - s_{ij}) \frac{\mathbf{p}_j - \mathbf{p}_i}{\|\mathbf{p}_j - \mathbf{p}_i\|}$$



- *Electrical repulsion* that node n_j exerts on n_i (repulsion strength r_{ij})

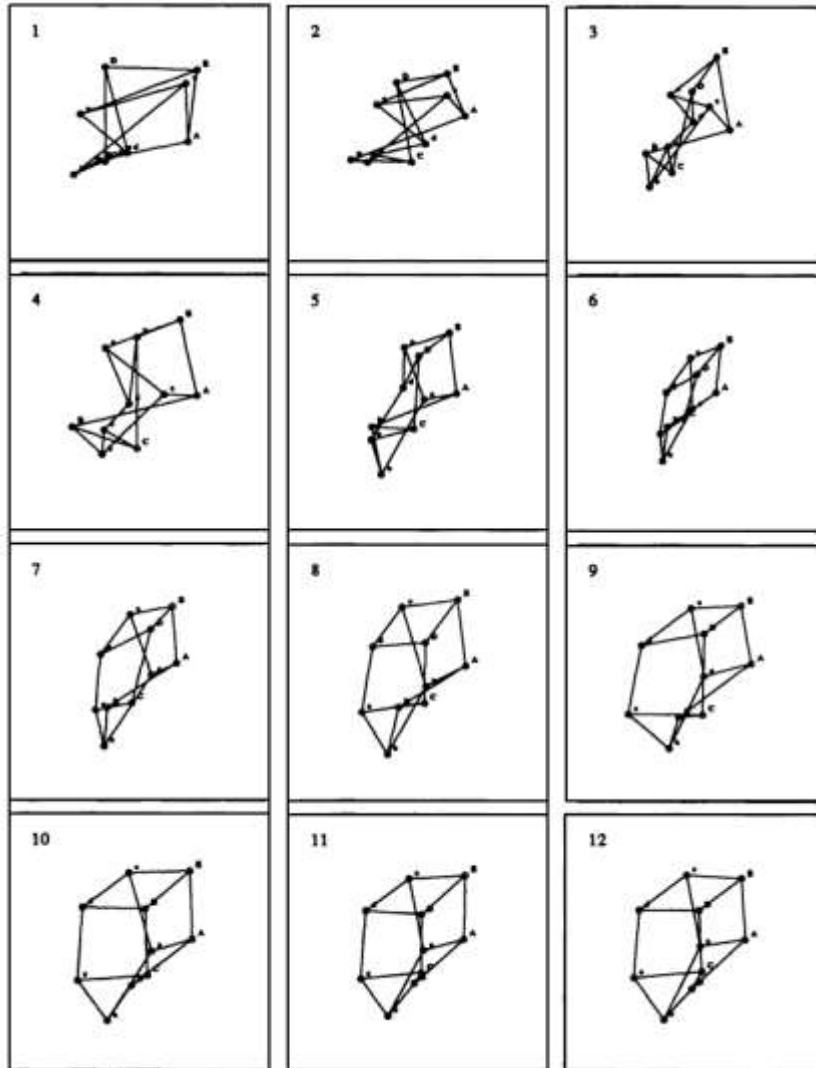
$$\mathbf{g}(\mathbf{p}_i, \mathbf{p}_j) = - \frac{r_{ij}}{\|\mathbf{p}_j - \mathbf{p}_i\|^2} \frac{\mathbf{p}_j - \mathbf{p}_i}{\|\mathbf{p}_j - \mathbf{p}_i\|}$$

- **Quiz:** Do you see potential problems with this?

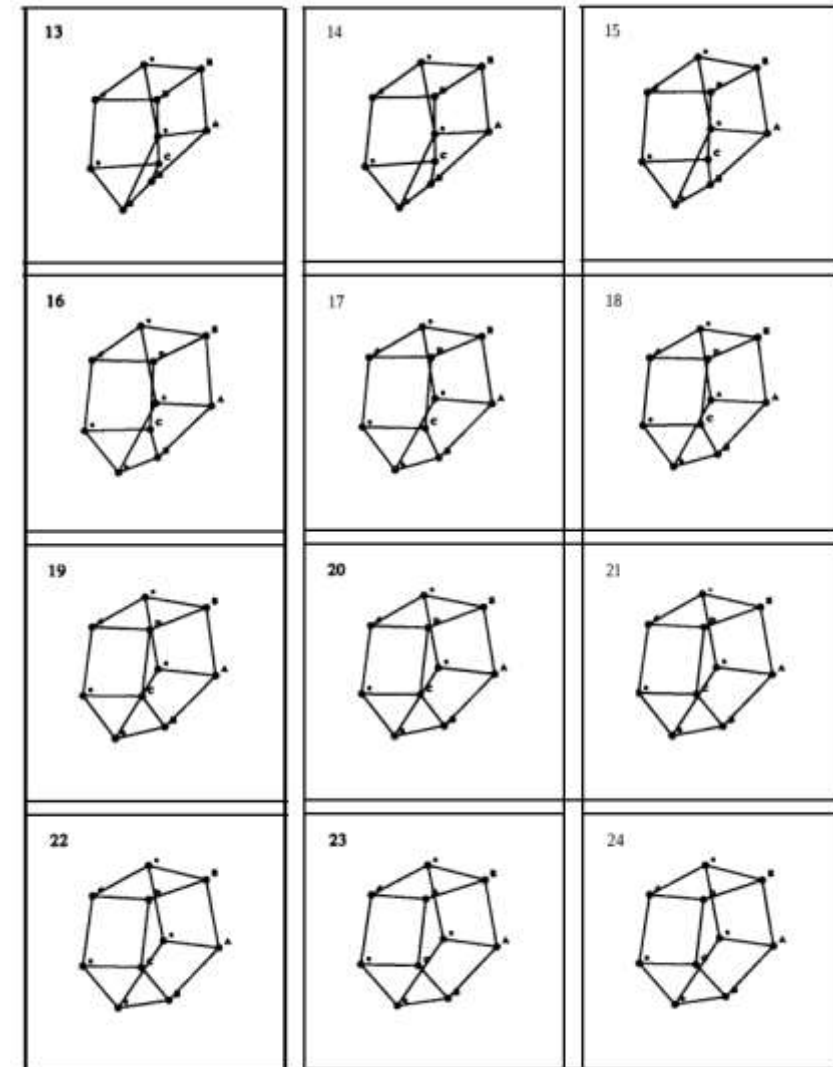
Force-Directed Layouts: General Algorithm

- Initialize (randomly or through a heuristic)
- Iterate:
 - For each node:
 - Sum all attractive and repulsive forces
 - Multiply overall force by stepsize (“temperature”)
 - Impose a maximum displacement (e.g., keep within image)
 - Move node
 - Adjust temperature

Quenching and Simmering



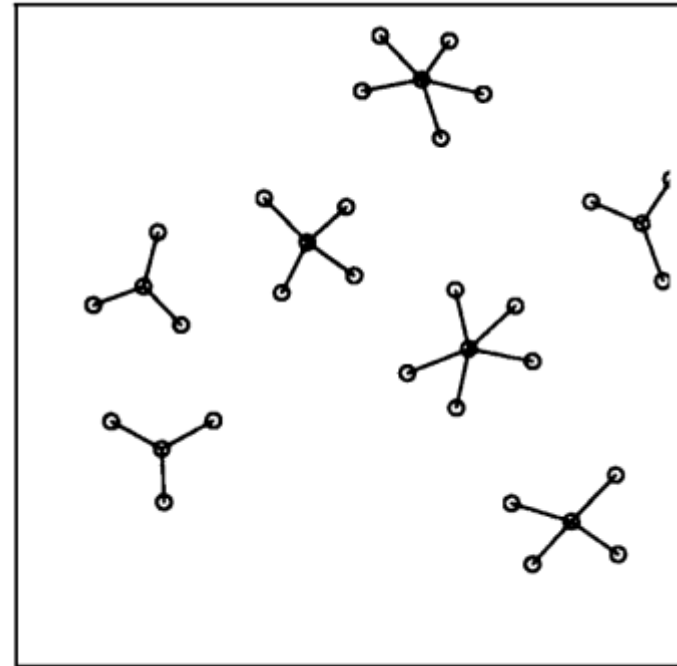
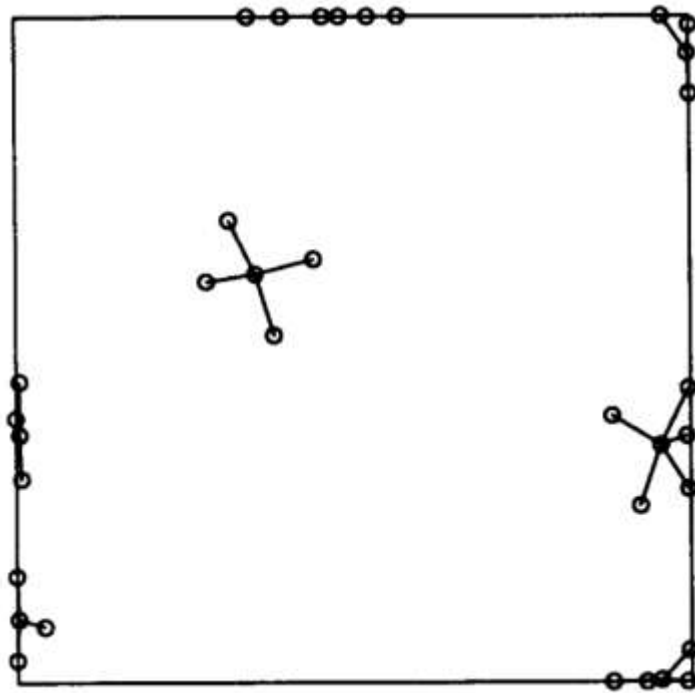
Quenching: Rapid cooling



Simmering: Constant low temperature

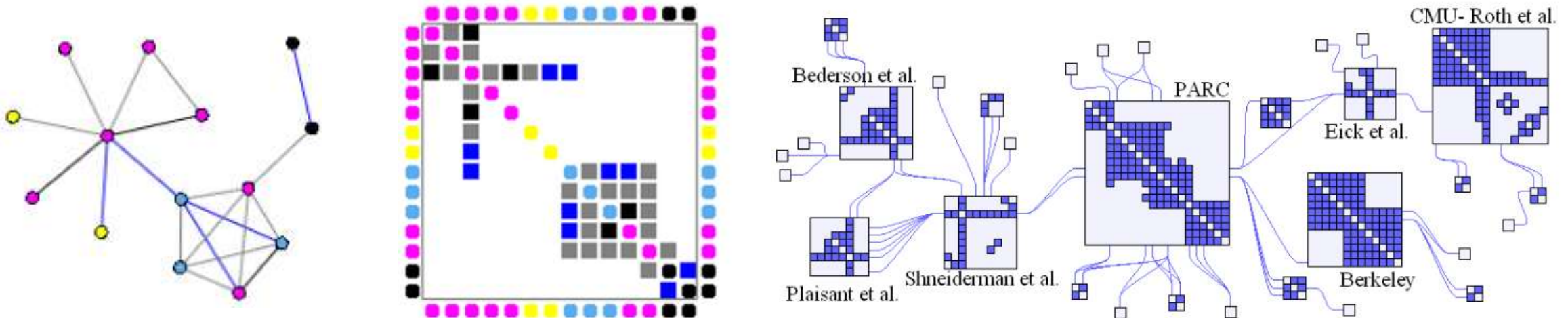
Grid Variant

- **Truncating** repelling forces beyond a certain distance produces more reasonable results for disconnected graphs
 - It also **speeds up** the algorithm, since we no longer need to consider all pairwise interactions



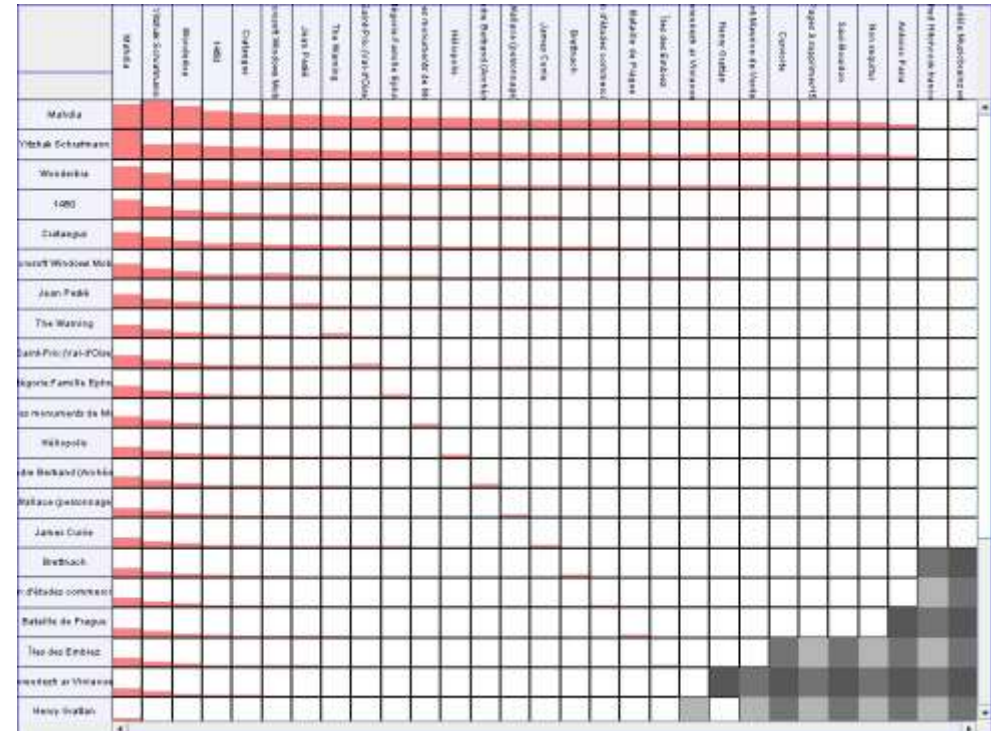
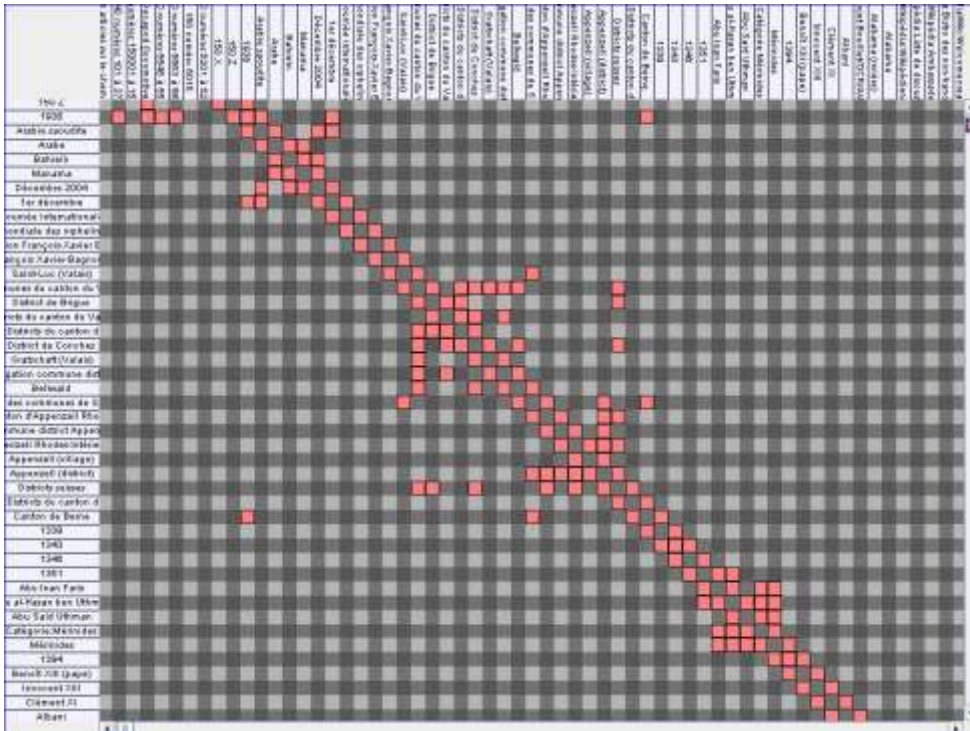
Node-Link Diagrams vs. Matrix Views

- Visualizing the **adjacency matrix** is the most popular alternative to node-link diagrams
 - Matrix views better suited for dense graphs
 - Readability depends on keeping communities together in vertex ordering
 - Node-link diagrams better suited for path following
- [Henry et al. 2007] combine benefits of both in a hybrid view



Matrix Views of Large Graphs

- [Elmqvist et al. 2008] implement a **zoomable adjacency matrix explorer (ZAME)** for large graphs
 - Vertices are grouped when zooming out
 - Edge are aggregated (e.g., average, range)

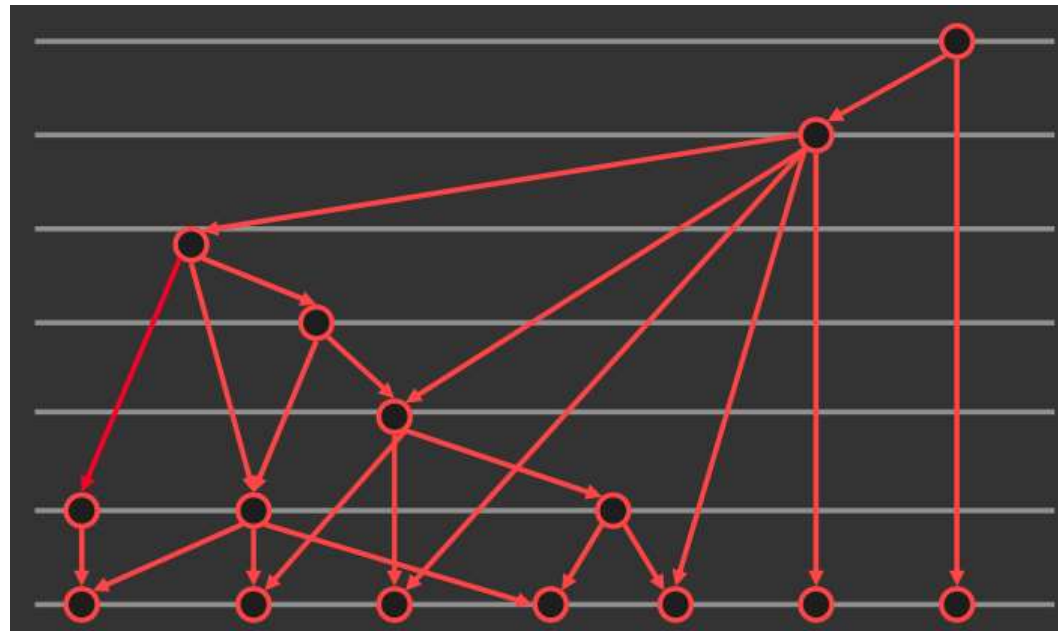


Summary: Node-link Diagrams and Matrix Views

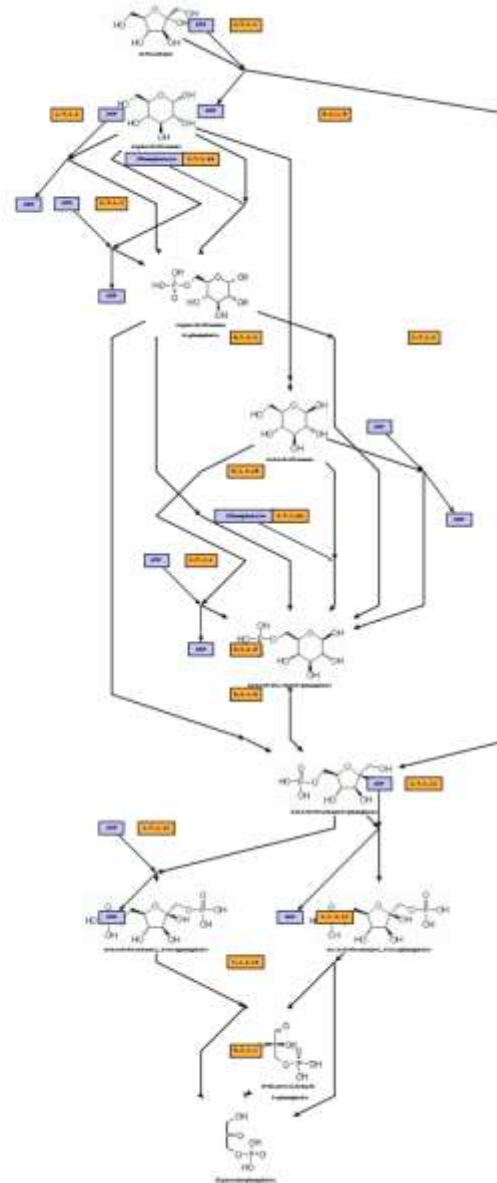
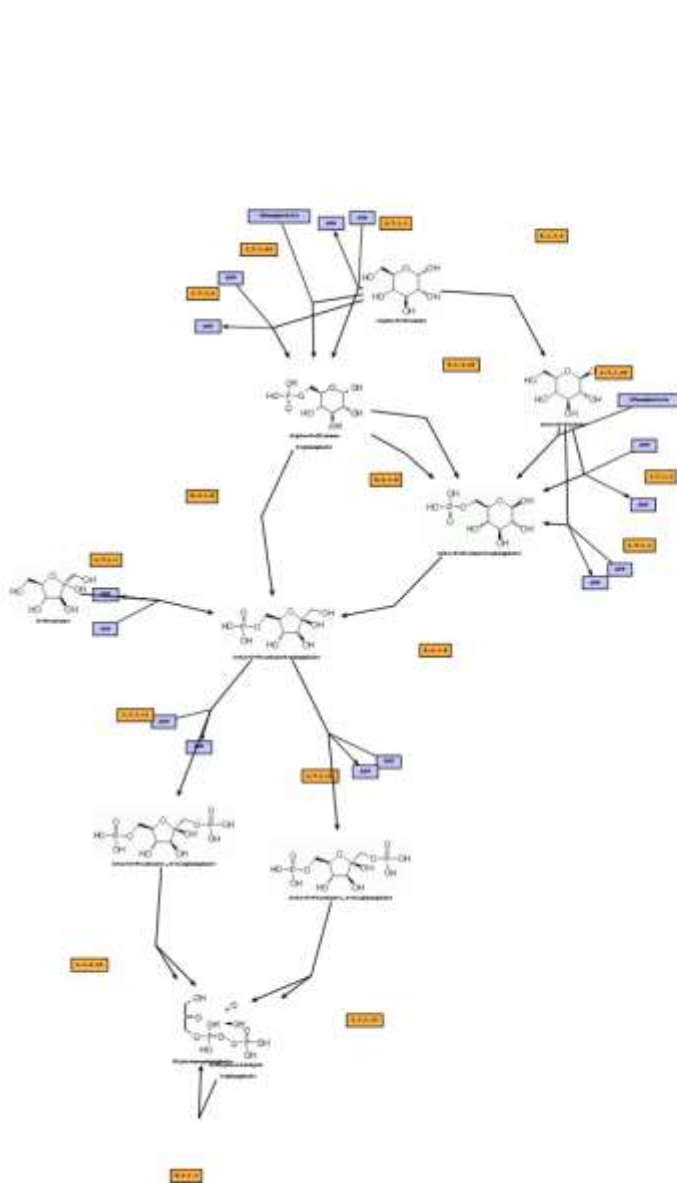
- **Node-link diagrams** via force-based methods are among the basic and widely used algorithms for graph layout
 - *Spring force*: Adjacent nodes strive for a constant distance
 - *Electrostatic force*: Nodes strive to avoid occlusion and to use available space
 - Common tricks for **faster and better results**:
 - Adjusting the temperature over iterations (“quenching and simmering”)
 - Neglecting electrostatic forces beyond some cutoff distance
- **Matrix views** are an alternative for dense and large graphs
 - Less clutter, scale to larger graphs
 - Less suited for path-related tasks

Appendix: Layered Layouts

- Nodes of **Directed Acyclic Graphs** can be organized in layers
 - Assign vertical position to each layer
 - Suggests a hierarchy, similar to tree visualization



Example: Metabolic Pathway



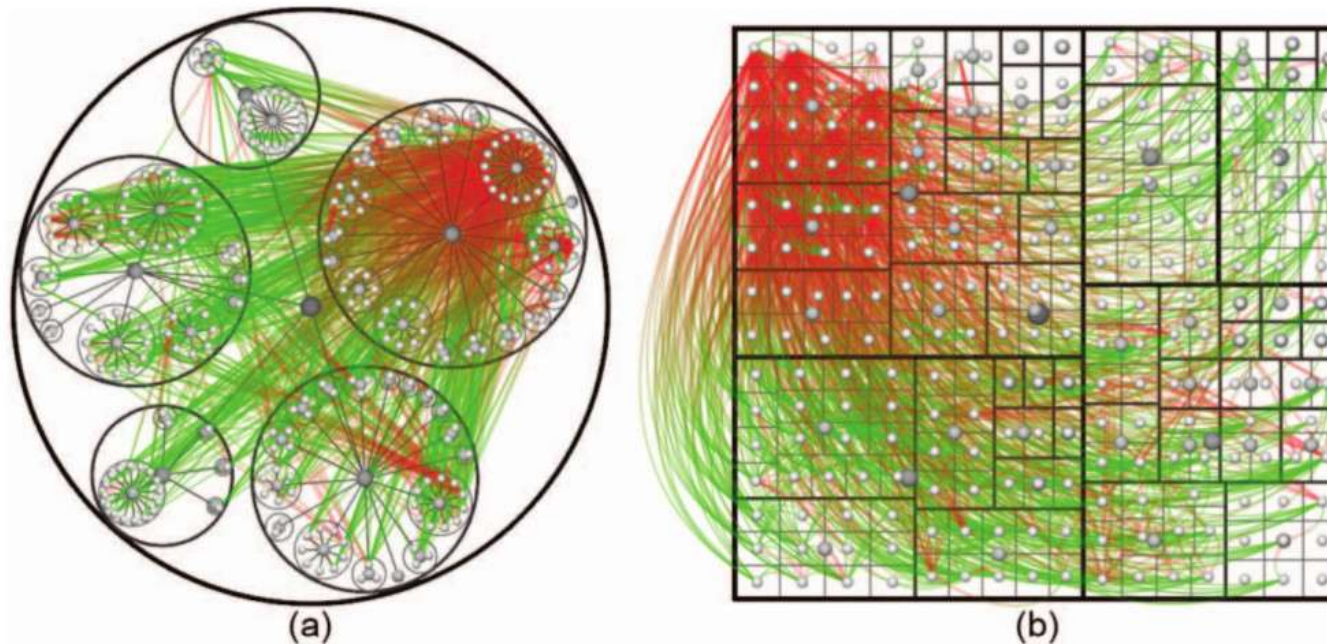
Left: Force-directed Layout

Right: Layered Layout

Section 5.3: Edge Bundling

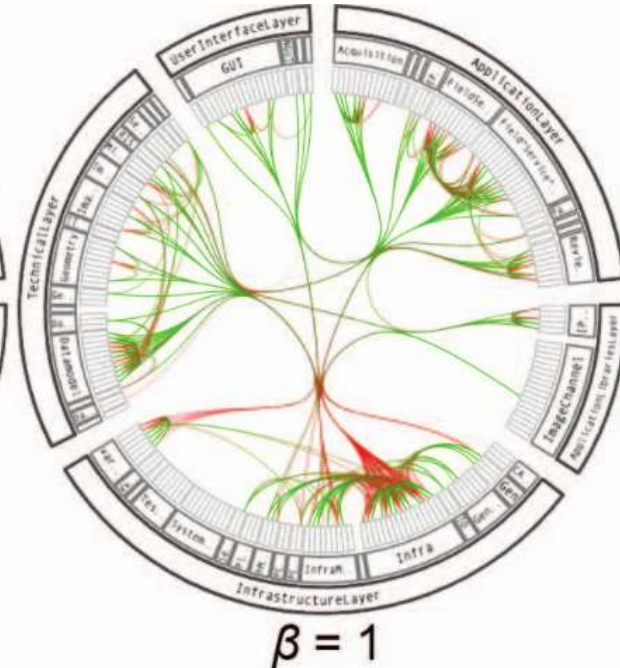
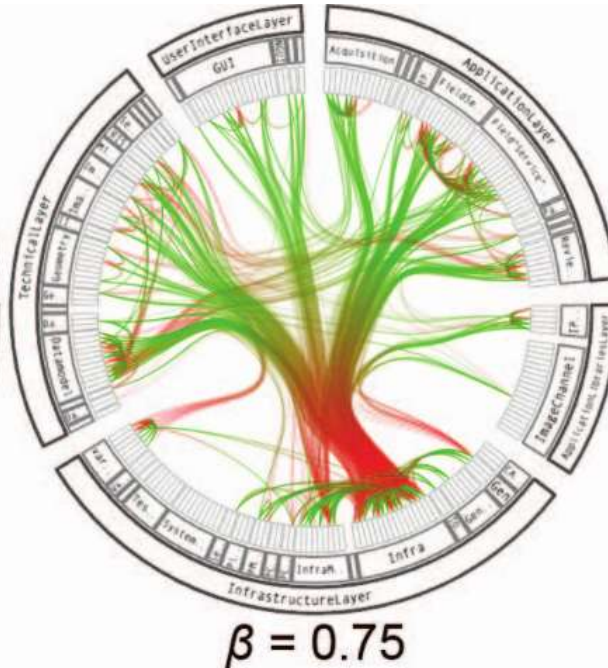
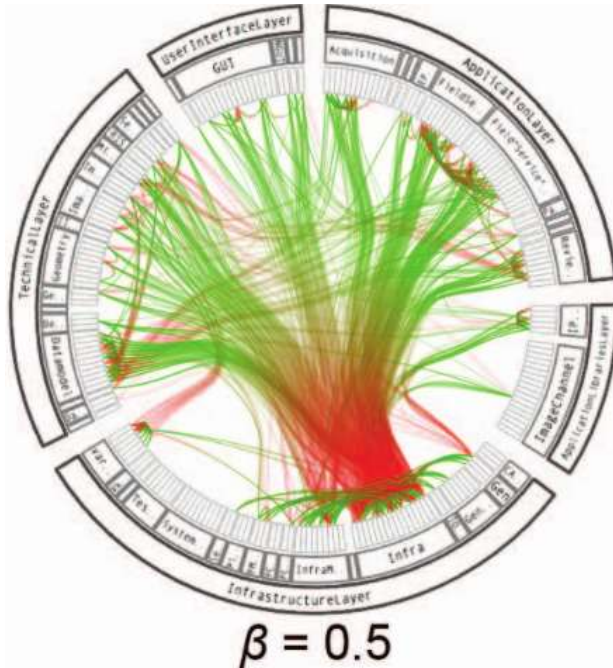
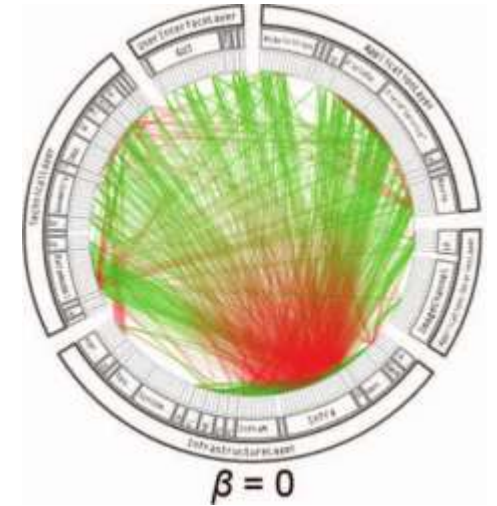
Motivation

- What if you have hierarchical (*inclusion*) and non-hierarchical (*adjacency*) relationships at the same time?
 - *Example:* software system; caller green, callee red



Idea: Edge Bundling

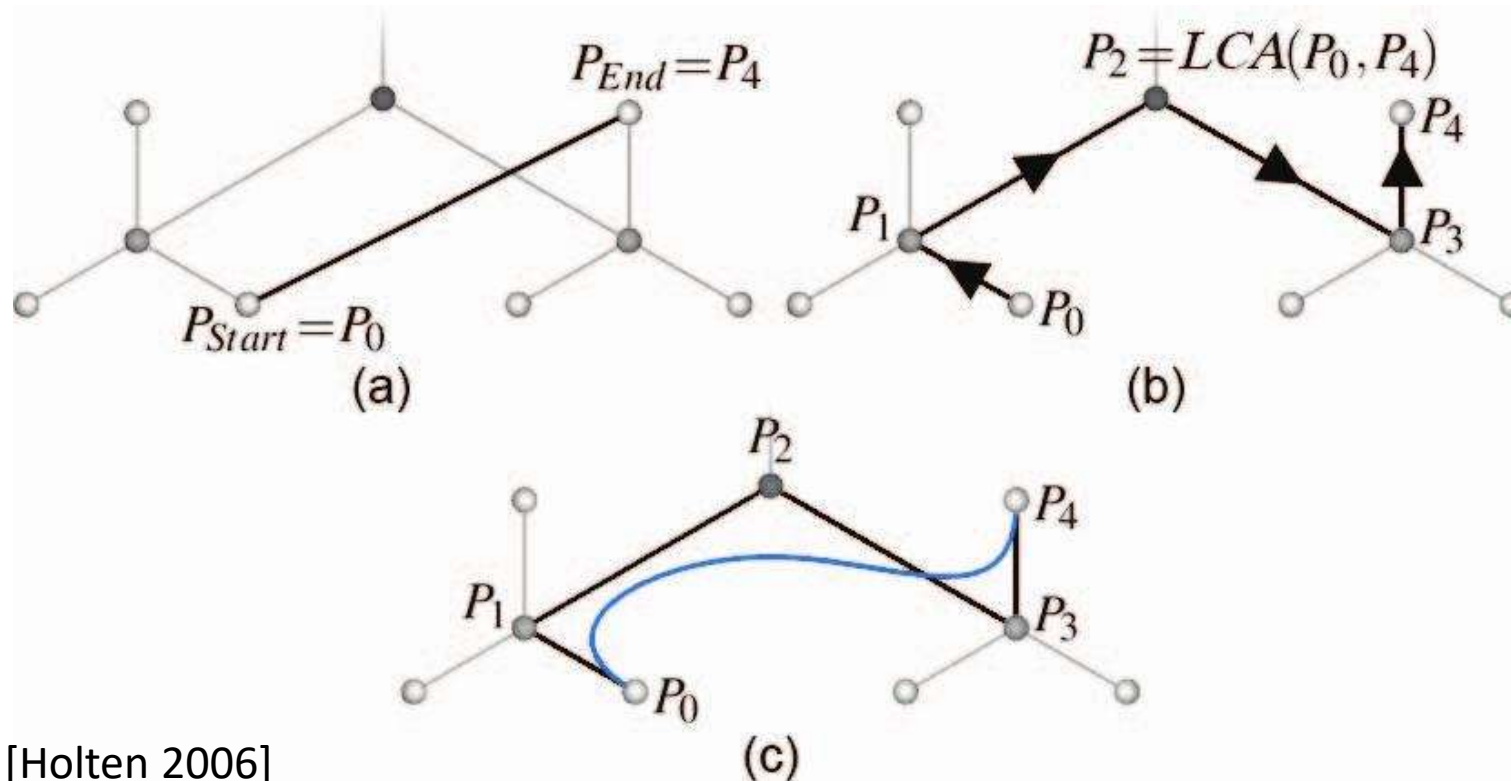
- Idea:** Bundle together edges that originate from or end in similar parts of the hierarchy



Hierarchical Edge Bundling: Concept

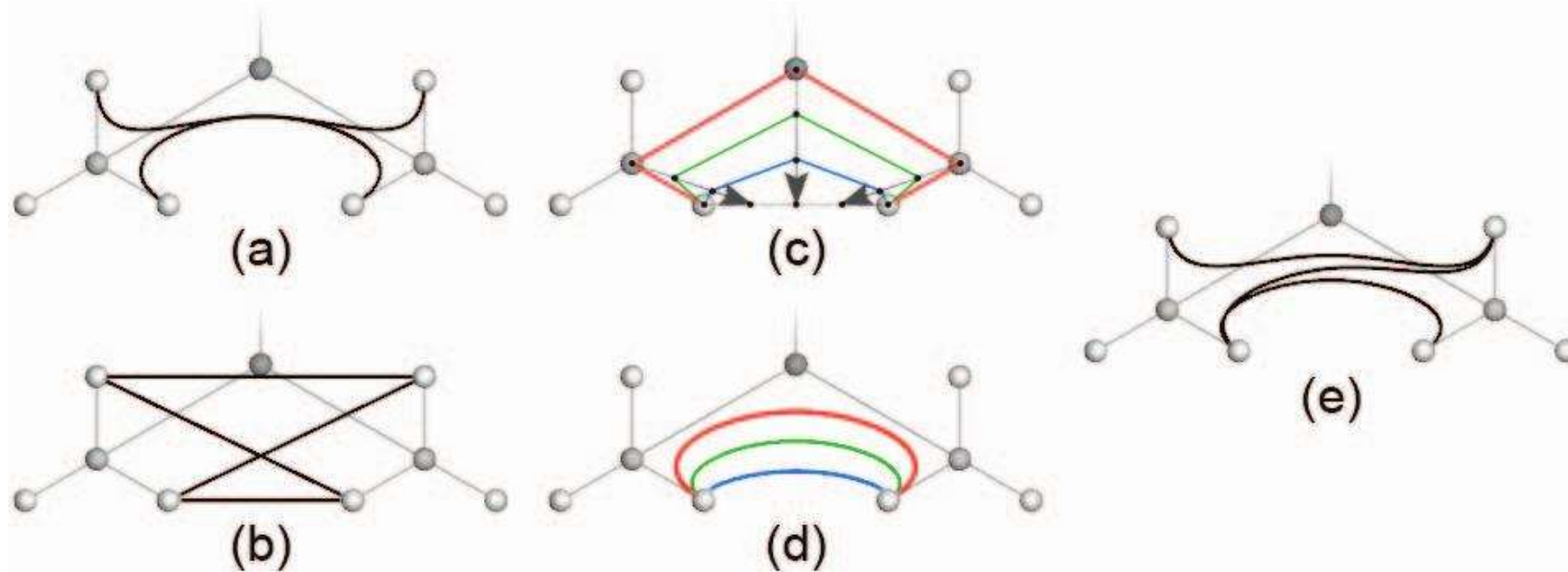
- **Idea:**

- Use standard tree layout for hierarchy
- Graph edges drawn as B-splines with path along tree as control polygon



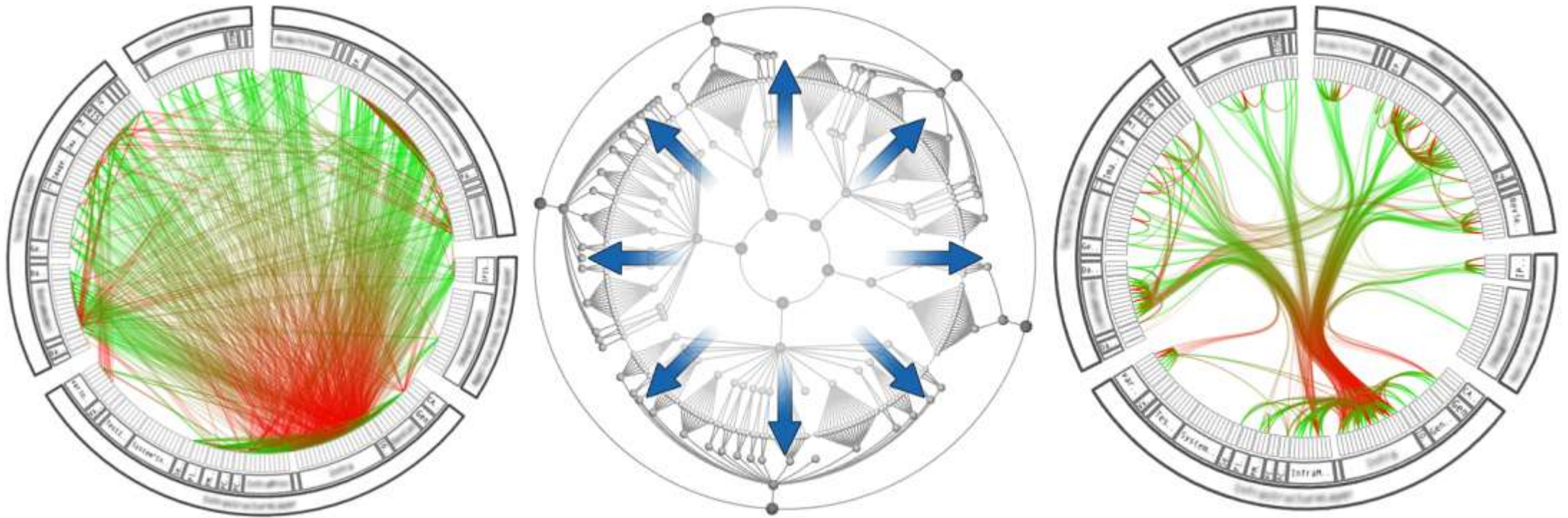
Edge Bundling Strength

- Bundling can lead to ambiguities
- Continuously straightening edges using a **bundling strength** $\beta \in [0,1]$ resolves this



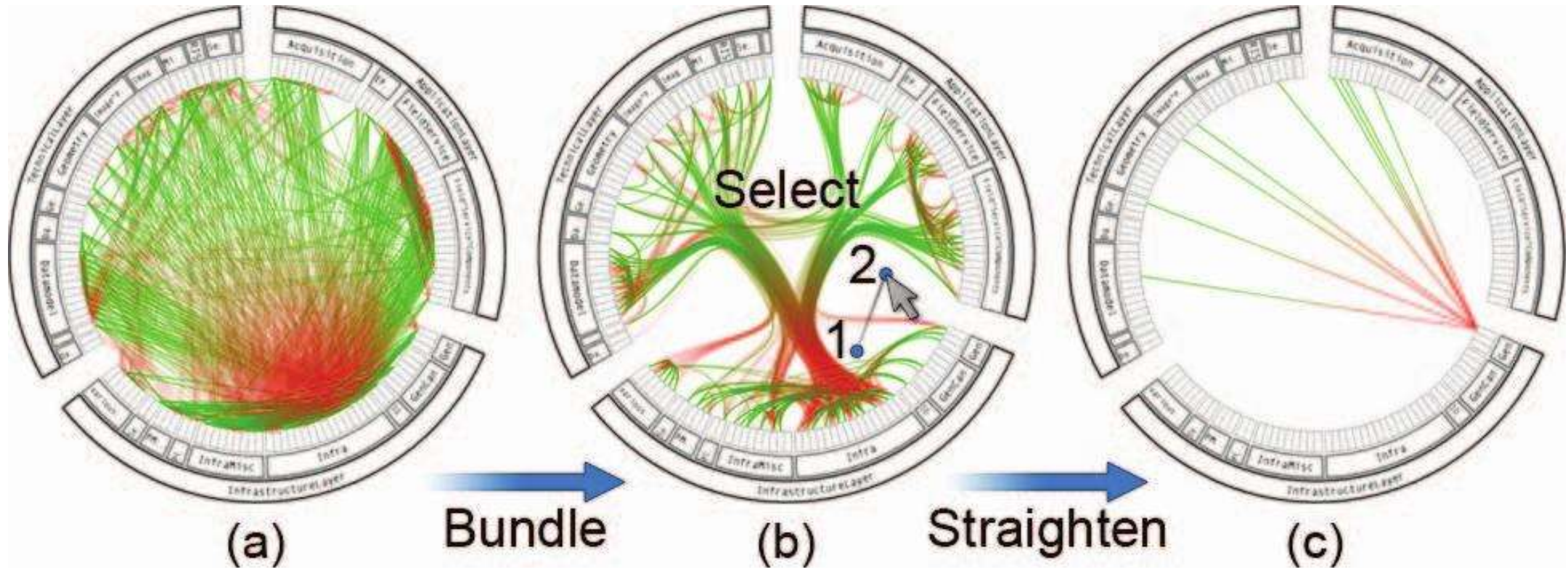
Edge Bundling: Radial Layout

- **Radial Layout** can be used for control polygons; nodes mirrored to the outside for labeling



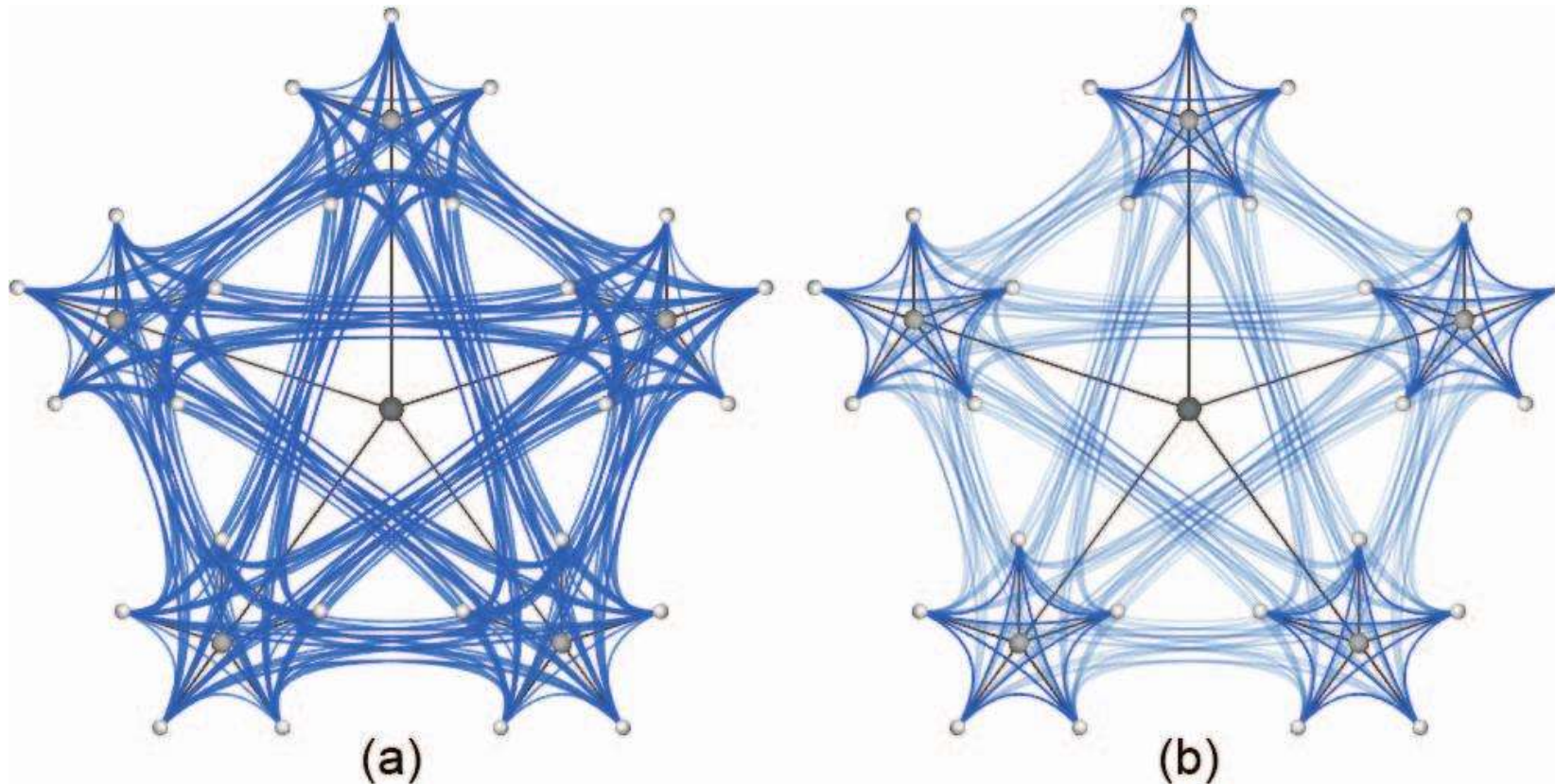
Edge Bundling: Interaction

- **Interactive filtering** and straightening helps with disambiguation:



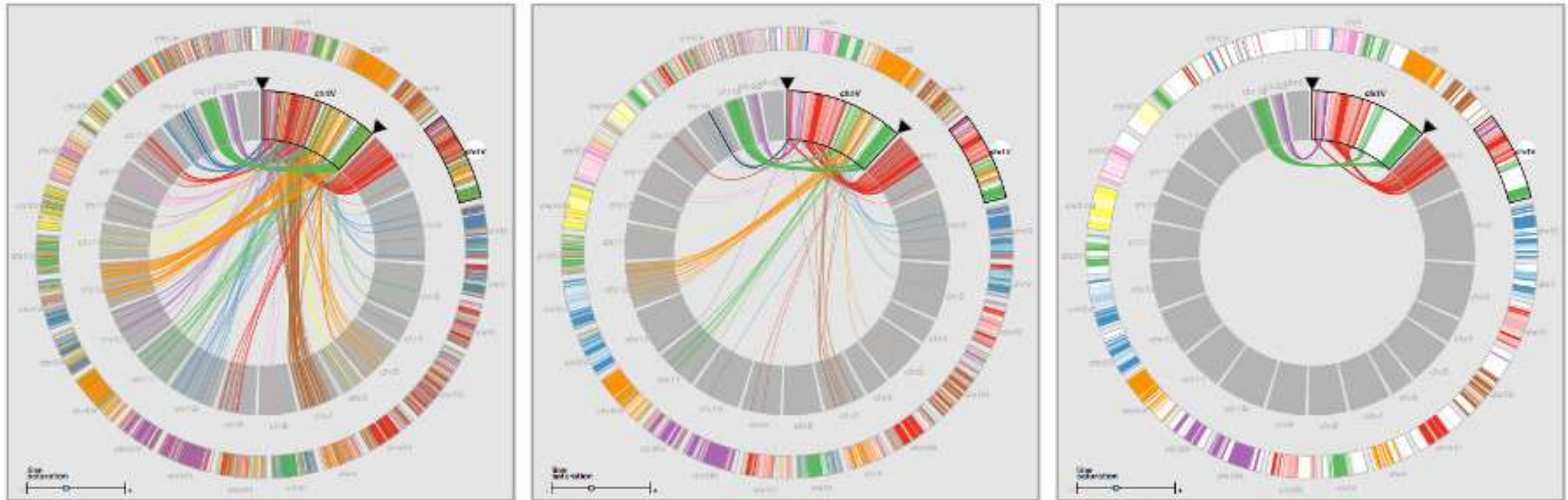
Edge Bundling: Transparency

- Making longer bundles more **transparent** allows us to more easily see shorter edges



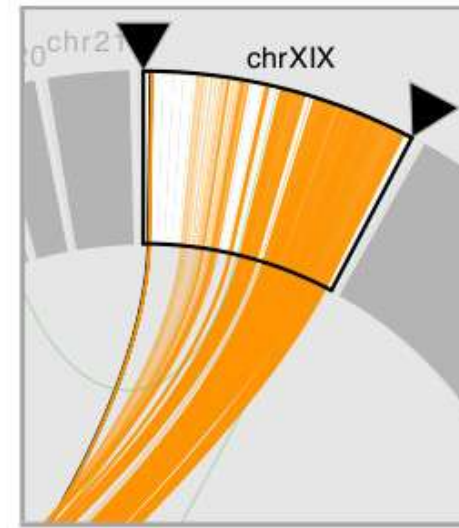
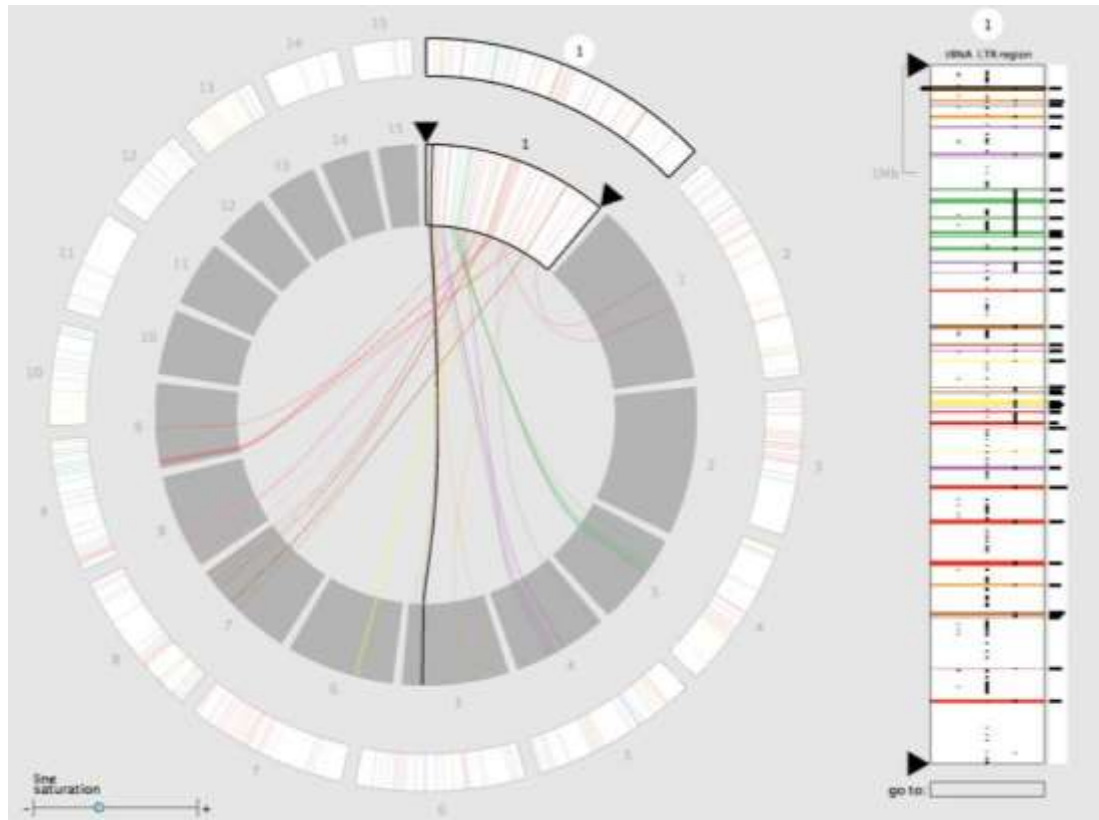
Graph Visualization for Comparative Genomics

- [Meyer et al. 2009]: Synteny (conservation of genomic features) between genomes

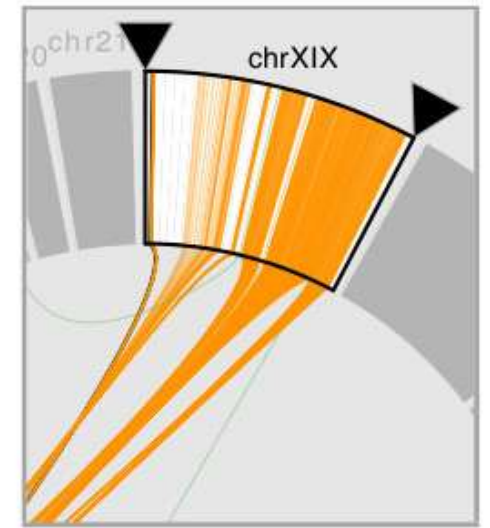


Edge Bundling in Comparative Genomics

MizBee [Meyer et al. 2009] bundles edges from the same source block that are preserved in the same target chromosome



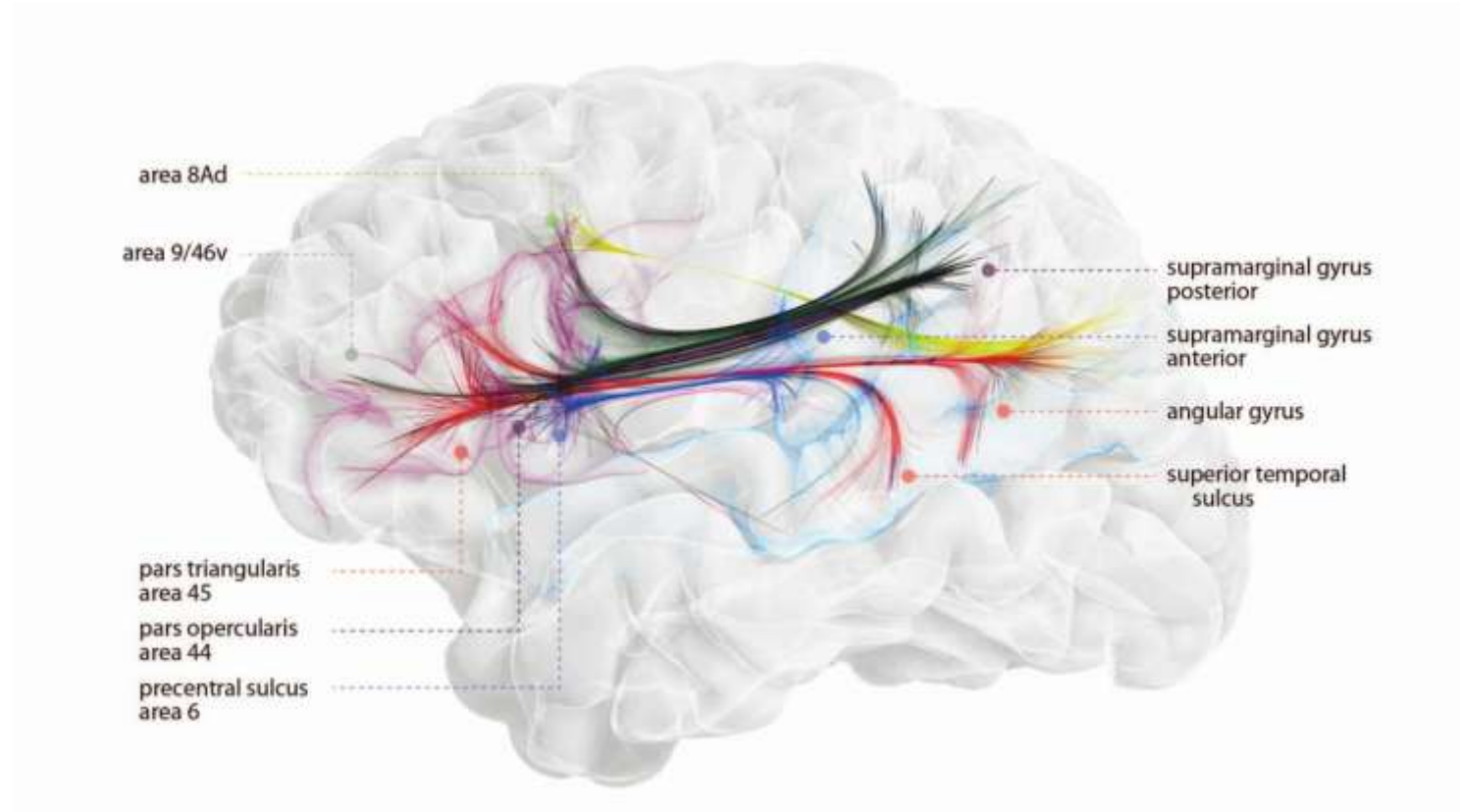
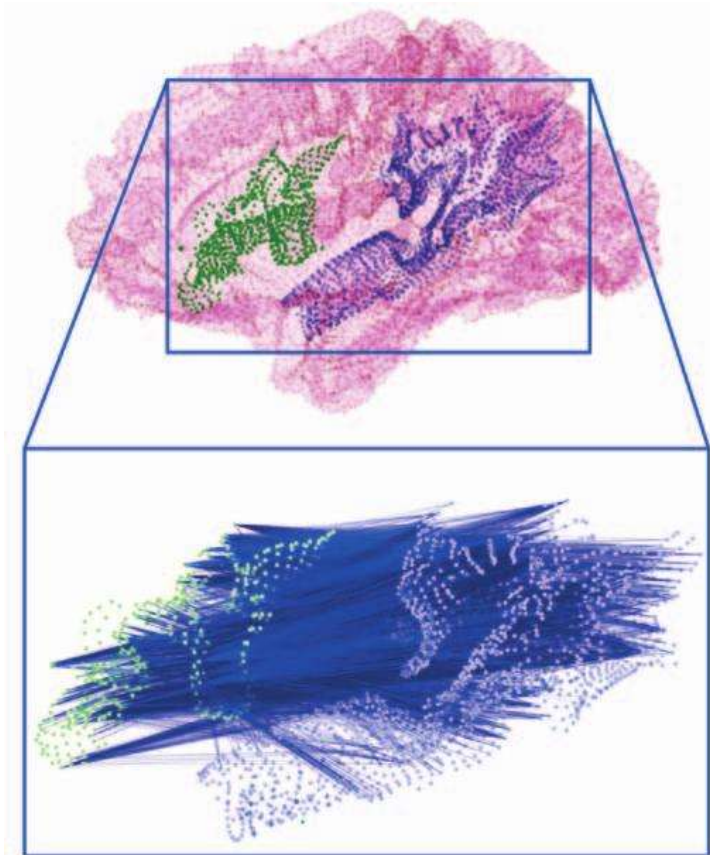
(a)



(b)

Edge Bundling in Neuroimaging

[Böttger et al. 2014] iteratively move similar edges in 3D fMRI connectivity graphs towards each other



Summary: Edge Bundling

- **Edge Bundling** draws edges as curves rather than straight line segments, so that similar edges “bundle together”
 - Original use case: Combining hierarchical and non-hierarchical information
 - Use path on tree layout as a control polygon for graph edge
 - Reducing bundling strength and opacity can reduce ambiguities
 - Other uses in comparative genomics and neuroimaging

References Methods

- Matthew O. Ward, Georges Grinstein, Daniel Keim. **Interactive Data Visualization: Foundations, Techniques, and Applications.** A. K. Peters/CRC Press, 2010
- Roberto Tamassia (Ed). **Handbook of Graph Drawing and Visualization.** CRC Press, 2013
- Danny Holten. **Hierarchical Edge Bundles: Visualization of Adjacency Relations in Hierarchical Data.** IEEE Trans. on Visualization and Computer Graphics 12(5):741-748, 2006

References Applications

- Miriah D. Meyer, Tamara Munzner, Hanspeter Pfister. **MizBee: A Multiscale Synteny Browser**. IEEE Trans. Vis. Comput. Graph. 15(6): 897-904 (2009)
- Joachim Böttger, Alexander Schäfer, Gabriele Lohmann, Arno Villringer, Daniel S. Margulies. **Three-Dimensional Mean-Shift Edge Bundling for the Visualization of Functional Connectivity in the Brain**. IEEE Trans. Vis. Comput. Graph. 20(3): 471-480 (2014)

Exam-Like Questions

1. What is a thread in the context of the Reingold-Tilford algorithm, and what is it used for?
2. State a benefit and a drawback of treemap visualizations.
3. What are two advantages of truncating the repelling force in a force-based graph layout at some distance?
4. Describe an ambiguity that can arise when using edge bundling, and a way to overcome it.